PRESIDENCY UNIVERSITY

Private University Estd. in Karnataka State by Act No. 41 of 2013

Itgalpura, Rajankunte, Yelahanka, Bengaluru – 560064

AGENTIC MULTI-TENANT MDR FRAMEWORK FOR AI-ASSISTED SECURITY OPERATIONS

AN INTERNSHIP REPORT

Submitted by

HIDA FATHIMA P H - 20221CAI0002

Under the guidance of,

Dr. VIJAYAKUMAR ADAICKALAM

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

(Artificial Intelligence & Machine Learning)

GAIN MORE KNOWLEDGE
REACH GREATER HEIGHTS

PRESIDENCY UNIVERSITY

BENGALURU

APRIL 2026

PRESIDENCY UNIVERSITY

Private University Estd. in Karnataka State by Act No. 41 of 2013

Itgalpura, Rajankunte, Yelahanka, Bengaluru – 560064

AGENTIC MULTI-TENANT MDR FRAMEWORK FOR AI-ASSISTED SECURITY OPERATIONS

AN INTERNSHIP REPORT

Submitted by

HIDA FATHIMA P H - 20221CAI0002

Under the guidance of,

Dr. VIJAYAKUMAR ADAICKALAM

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

(Artificial Intelligence & Machine Learning)

GAIN MORE KNOWLEDGE
REACH GREATER HEIGHTS

PRESIDENCY UNIVERSITY

BENGALURU

APRIL 2026

PRESIDENCY UNIVERSITY

Private University Estd. in Karnataka State by Act No. 41 of 2013
Itgalpura, Rajankunte, Yelahanka, Bengaluru – 560064

PRESIDENCY SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

BONAFIDE CERTIFICATE

Certified that this report **"Agentic Multi-Tenant MDR Framework For AI-Assisted Security Operations"** is a bonafide work of **HIDA FATHIMA P H (20221CAI0002),** who has successfully carried out the internship work and submitted the report for partial fulfillment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE AND ENGINEERING (SPECIALIZATION DIVISION), ARTIFICIAL INTELLIGENCE & MACHINE LEARNING during 2025-2026.

20/4/26

Dr. Vijayakumar Adaickalam
Professor
Internship Supervisor
PSCS (Spl. Division)
Presidency University

20/04/2026

Mr. Anandan Bellie
Assistant Professor
Program Internship Coordinator
PSCS (Spl. Division)
Presidency University

20/4/26

Dr. Geetha A
Associate Professor
School Internship Coordinator
PSCS (Spl. Division)
Presidency University

20/04/2026

Dr. Zafar Ali Khan N
Professor & Head of the Department
PSCS (Spl. Division)
Presidency University

Dr. Shakkeera L
Associate Dean
PSCS (Spl. Division)
Presidency University

Sl. No	Name of the Examiner	Signature
1.	MOHAMMADI AKHEELA KHANUM	21.4.26
2	Dr. Prajwalesh C.B	

PRESIDENCY SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

DECLARATION

I am a student of final year B.Tech in COMPUTER SCIENCE AND ENGINEERING (ARTIFICIAL INTELLIGENCE & MACHINE LEARNING), at Presidency University, Bengaluru, named **Hida Fathima P H**, hereby declare that the internship work titled **"Agentic Multi-Tenant MDR Framework For AI-Assisted Security Operations"** has been independently carried out by me and submitted in partial fulfillment for the award of the degree of B.Tech in COMPUTER SCIENCE ENGINEERING (Spl. Division) (ARTIFICIAL INTELLIGENCE & MACHINE LEARNING) during the academic year of 2025-2026. Further, the matter embodied in the internship has not been submitted previously by anybody for the award of any Degree or Diploma to any other institution.

Hida Fathima P H USN: 20221CAI0002 Hida

PLACE: BENGALURU

DATE: 20 April 2026

INTERNSHIP COMPLETION CERTIFICATE

Tecplix
WHERE IDEAS THRIVE

+91 6636 600 700
www.tecplix.com
info@tecplix.com

Date: April 10, 2026

Subject: Internship Experience Letter

TO WHOMSOEVER IT MAY CONCERN

This letter is to certify that **Ms. Hida Fathima PH** was working as Intern - AI and Cyber Security with Tecplix Technologies Pvt. Ltd. Her internship tenure was from January 05, 2026 to April 03, 2026.

We wish her all the best for her future endeavors.

For Tecplix Technologies Pvt Ltd

TECPLIX TECHNOLOGIES PVT. LTD.

Sahana P S
Sr. Vice President-HR

Tecplix Technologies Pvt. Ltd.
#142, ORR, Near HSR Flyover, Sector 5, HSR Layout, Bangalore, Karnataka, India - 560 102
CIN: U72900KA2019PTC126311

ACKNOWLEDGEMENTS

For completing this internship work, I have received the support and the guidance from many people whom I would like to mention with deep sense of gratitude and indebtedness. I extend my gratitude to our beloved **Chancellor, Pro-Vice Chancellor, and Registrar** for their support and encouragement in completion of the internship.

I would like to sincerely thank my internal Supervisor **Dr. Vijayakumar Adaickalam**, Professor, Presidency School of Computer Science and Engineering (Spl. Division), Presidency University, for his moral support, motivation, timely guidance and encouragement provided to me during the period of internship work.

I am also thankful to **Dr. Zafar Ali Khan N,** Professor, Head of the Department - CAI, Presidency School of Computer Science and Engineering (Spl. Division)**,** Presidency University, for his mentorship and encouragement.

I express my cordial thanks to **Dr. Shakkeera L**, Associate Dean, Presidency School of Computer Science and Engineering (Spl. Division), and also the Management of Presidency University for providing the required facilities and intellectually stimulating environment that aided in the completion of my internship work.

I am grateful to **Dr. Geetha A,** PSCS (Spl. Division), School Internship Coordinator and **Mr. Anandan Bellie,** Program Internship Coordinator**,** Presidency School of Computer Science and Engineering (Spl. Division), for facilitating problem statements, coordinating reviews, monitoring progress, and providing their valuable support and guidance.

I am also grateful to Teaching and Non-Teaching staff of Presidency School of Computer Science and Engineering and also staff from other departments who have extended their valuable help and cooperation.

Hida Fathima P H

ABSTRACT

The current problem that Security Operations Centers (SOCs) are grappling with is a huge data issue. Every day, modern enterprise networks are producing thousands of security alerts, and the human analysts cannot possibly go through them all manually. During the course of this internship, as I learned about SOC workflows, it became apparent that there is a great deal of time wasted by analysts who have to switch between unrelated security monitoring tools. This swivel chair analysis has been repeated to cause severe alert fatigue and significantly slows down the speed at which a team can halt a real cyber-attack. Although numerous companies are trying to apply Large Language Models (LLMs) to accelerate the process of alert triage, the upload of sensitive company network logs to AI models in the public clouds poses unacceptable risks to data privacy. Moreover, common LLMs also hallucinate fabricated facts, which is highly perilous during incident response.

The proposed project proposes the Agentic Multi-Tenant Managed Detection and Response (MDR) Framework to address these practical problems. The system was configured as a dockerized microservice environment that does not use conventional chatbot interfaces by deploying a 7-Agent AI Swarm which runs locally. Since, the LLM inferencing occurs on the host machine itself, there is no risk of corporate information being leaked into external servers. The platform reports live attacks to an external decoy honeypot sensor and feeds which are then fed to a localized SQLite data lake via zero-trust webhooks. The framework will support various security personnel members with a severe 14-persona Role-Based Access Control system, meaning that users can only interact with the particular AI agents and data as authorized by their job role. In the course of an investigation, the swarm employs Retrieval-Augmented Generation and local threat heuristics to process logs, identify technical artifacts, correlate attacks with the MITRE ATT&CK framework and generate formal SOAR incident reports automatically.

Experiments showed the division of the AI into specialized tasks and rigid restrictions effectively prevented the creation of fake data by the models. Finally, this framework offers a safe, operational remedy that vastly saves the Mean Time to Respond (MTTR) to threats, and yet preserves human analyst in the ultimate decision-making of security.

TABLE OF CONTENTS

Sl. No.	Title	Page No.
	Declaration	iii
	Internship Completion Certificate	iv
	Acknowledgement	v
	Abstract	vi
	List of Figures	ix
	List of Tables	xi
	Abbreviations	xii
1.	Introduction	1
	1.1 Background	1
	1.2 Statistics of Internship	2
	1.3 Prior Existing Technologies	3
	1.4 Proposed Approach	4
	1.5 Objectives	5
	1.6 SDGs	7
	1.7 Overview of Internship Report	8
2.	Literature Review	10
3.	Methodology	15
	3.1 Data Ingestion and Normalization Methodology	15
	3.2 AI Routing and Agent Swarm Engineering	17
	3.3 Mathematical Grounding Methodology	18
4.	Project Management	20
	4.1 Internship Timeline	20
	4.2 Risk Analysis	22
	4.3 Internship Operational Requirements	24
5.	Analysis and Design	26
	5.1 Requirements	26
	5.2 Block Diagram	27
	5.3 System Flow Chart	28
	5.4 Choosing Devices	29

5.5 Designing Units 30
5.6 Standards 32
5.7 Mapping with IoTWF Reference Model Layers 32
5.8 Domain Model Specification 35
5.9 Communication Model 36
5.10 Cloud / Enterprise Deployment Level 36
5.11 Functional View 37
5.12 Mapping Deployment Level with Functional View 37
5.13 Operational View 38
5.14 Other Design 38
6. Hardware, Software and Simulation 40
6.1 Hardware 40
6.2 Software Development Tools 41
6.3 Software Code 43
6.4 Simulation 44
7. Evaluation and Results 47
7.1 Test Points 47
7.2 Test Plan 48
7.3 Test Results 48
7.4 Insights 51
8. Social, Legal, Ethical, Sustainability and Safety Aspects 54
8.1 Social Aspects 54
8.2 Legal Aspects 55
8.3 Ethical Aspects 56
8.4 Sustainability Aspects 56
8.5 Safety Aspects 57
9. Conclusion 59
References 61
Base Paper 64
Appendix A: Pseudocode 65
Appendix B: Screenshots 69

LIST OF FIGURES

Sl. No.	Figure Name	Caption	Page No.
1	Figure 1.1	United Nations Sustainable Development Goals (SDGs)	7
2	Figure 3.1	Data Ingestion and Intelligence Integration Architecture	16
3	Figure 3.2	Workflow of the Multi-Agent MDR System	17
4	Figure 4.1	Project SDLC Timeline	21
5	Figure 5.1	High-Level Block Diagram of the Agentic MDR Architecture	28
6	Figure 5.2	Sequential Data Flow and Alert Lifecycle within the Framework	29
7	Figure 5.3	Mapping the Agentic MDR Architecture to the IoTWF 7-Layer Reference Model	33
8	Figure 5.4	Authentication, RBAC Enforcement, and Persona-Specific Operational Paths in the MDR System	39
9	Figure 6.1	Docker Container Architecture of the Agentic MDR Framework	42
10	Figure 6.2	Simulated Attack Workflow and Ingestion Sequence	44
11	Figure 7.1	Simulated Output: AI Swarm analysis and MITRE mapping of the Port Scan attack	49
12	Figure 7.2	Simulated Output: Automated investigation of an SQL Injection payload	50
13	Figure 7.3	Simulated Output: Agentic analysis and remediation strategy for an SSH Brute Force attack	51
14	Figure B.1	Simulated Output: The Platform Administration dashboard	69
15	Figure B.2	Simulated Output: The Live Telemetry Stream interface	69
16	Figure B.3	Simulated Output: The Secure Evidence Locker	70
17	Figure B.4	Simulated Output: A raw view of the backend audit.log file	70
18	Figure B.5	Simulated Output: The Orchestrator, Query, and Intel	71

		agents collaborating to investigate an alert	
19	Figure B.6	Simulated Output: An automated Incident Response Report	71
20	Figure B.7	Simulated Output: The Detection Agent autonomously generating rules	72
21	Figure B.8	Simulated Output: An automated investigation into a suspicious scheduled task creation	72

LIST OF TABLES

Sl. No.	Table Name	Table Caption	Page No.
1	Table 1.1	Common Security Telemetry Sources in a Modern SOC	2
2	Table 6.1	Core Software Libraries and Engineering Functions	43

ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
APT	Advanced Persistent Threat
ASGI	Asynchronous Server Gateway Interface
AWS	Amazon Web Services
CoT	Chain of Thought
CPU	Central Processing Unit
CSS	Cascading Style Sheets
CTI	Cyber Threat Intelligence
DoS	Denial of Service
EDR	Endpoint Detection and Response
FQL	Falcon Query Language
GCP	Google Cloud Platform
GPU	Graphics Processing Unit
GTI	Global Threat Intelligence
HITL	Human-in-the-Loop
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
IAM	Identity and Access Management
IDS	Intrusion Detection System
IoCs	Indicators of Compromise
IoTWF	Internet of Things Workflow
IOPS	Input/Output Operations Per Second
IP	Internet Protocol
IPS	Intrusion Prevention System
ISO	International Organization for Standardization
JSON	JavaScript Object Notation
KQL	Kusto Query Language
LLM	Large Language Model

LLMs	Large Language Models
MCP	Model Context Protocol
MDR	Managed Detection and Response
MITRE ATT&CK	MITRE Adversarial Tactics, Techniques, and Common Knowledge
MSSPs	Managed Security Service Providers
NER	Named Entity Recognition
NIST	National Institute of Standards and Technology
NLP	Natural Language Processing
OSI	Open Systems Interconnection
OSINT	Open Source Intelligence
OSV	Open Source Vulnerabilities
PII	Personally Identifiable Information
RAG	Retrieval-Augmented Generation
RBAC	Role-Based Access Control
SDK	Software Development Kit
SDG	Sustainable Development Goals
SDLC	Software Development Life Cycle
SecOps	Security Operations
SIEM	Security Information and Event Management
SLMs	Small Language Models
SOAR	Security Orchestration, Automation, and Response
SOC	Security Operations Center
SPA	Single Page Application
SQL	Structured Query Language
SSD	Solid State Drive
TCP/IP	Transmission Control Protocol / Internet Protocol
TLS	Transport Layer Security
UDM	Unified Data Model
UI	User Interface
VAPT	Vulnerability Assessment and Penetration Testing
XDR	Extended Detection and Response
XSS	Cross-Site Scripting

Chapter 1

INTRODUCTION

1.1 Background

There has been a significant shift in how corporate infrastructure is protected as enterprises have extended into the cloud and adopted distributed remote work concepts. Previously, the enterprise network was confined to a physical building and protected solely by a perimeter firewall. However, this conventional perimeter no longer exists. Corporations now rely on multiple security measures, physical, software-based, and cloud-based, as networks are heavily distributed across endpoints and multi-cloud environments, creating a vast and vulnerable attack surface.

Cyberattacks are no longer simply opportunistic malware; they are now highly organized, stratified infection campaigns executed by Advanced Persistent Threat (APT) groups. Instead of using easily detectable malware, attackers consistently utilize "living off the land" techniques and stealthy tactics, such as lateral network movement, silent privilege escalation, and credential theft. These tactics allow them to camouflage their activities within the company's everyday network traffic [1].

To effectively detect and defend against these advanced and evasive threats, organizations must continuously maintain visibility over all assets connected to their network. To monitor endpoint devices at the host level, they use security frameworks like Endpoint Detection and Response (EDR), which track activities on individual devices and servers. Simultaneously, cloud telemetry data lakes monitor broader network traffic and identity access logs [2], [3]. A structured log is generated for every minute activity, such as a file download, a registry modification, a login attempt, or a firewall block. To analyze and derive insights from these logs, the data is dispatched to a Security Information and Event Management (SIEM) system. Although modern engineering advancements have solved the problem of centralizing these logs into a single platform, this has aggravated the human capacity shortfall required to analyze such vast amounts of data. The primary sources of telemetry flowing into the SOC, and the purposes they serve during investigations, are outlined in

Table 1.1 below.

Table 1.1 Common Security Telemetry Sources in a Modern SOC

Telemetry Source	**Primary Function**	**Example Log Output**
EDR (Endpoint)	Monitors host-level activity.	Process execution, file modifications.
SIEM (Network)	Correlates broad network traffic.	Firewall blocks, DNS request anomalies.
IAM (Identity)	Tracks user authentication.	Failed login attempts, MFA triggers.
Cloud Infra	Monitors cloud workloads.	AWS CloudTrail events, S3 bucket access.

1.2 Statistics of Internship

At the beginning of the internship, observations made within a live, high-traffic Security Operations Center (SOC) regarding the alert fatigue problem closely corresponded to established academic benchmarks [4]. Monitoring the SIEM closely revealed that the system was flooded with thousands of automated alerts every single day. Extremely sensitive heuristic detection rules, overlapping vendor signatures, and onerous volumes of non-critical network traffic all added to a false positive rate that frequently exceeded the 60% threshold [5].

Furthermore, operational time-tracking revealed the true cost of this data overload. Manually investigating a single alert took Tier 1 analysts an average of 15 to 25 minutes. This investigation process required the tedious swivel chair analysis technique, where analysts constantly moved between screens, including the SIEM dashboard, external threat intelligence feeds, and active directory logs, just to gather the basic context of the alert. This severe manual challenge has been heavily documented in recent studies concerning automated attack investigations [6].

It is physically impossible to expect human analysts to read and manually correlate millions of lines of log data every day, which eventually leads to the formation of a dangerous and unavoidable backlog. Within these accumulated logs, highly critical network intrusion attempts often go unnoticed, camouflaged in a pile of thousands of false positives. Without the use of automated triage intervention systems, industry standard performance metrics like Mean Time to Detect (MTTD) and Mean Time to Respond (MTTR) end up degrading and suffering heavily. High MTTR values imply that attackers are afforded the lateral dwell time they require to exfiltrate sensitive data or deploy ransomware across the network [7].

1.3 Prior Existing Technologies

The daily workflow within a Security Operations Center (SOC) continues to be structurally deficient in spite of the advanced SIEM and threat intelligence suites that organizations have employed. The sequence of incident response operations still relies entirely on manual human intervention, which proves to be a major flaw in the system. When a new threat is detected, security engineers must manually write highly specific detection rules using complex query languages like YARA, Kusto Query Language (KQL), and Falcon Query Language (FQL) to hunt down the attackers across the network [2].

A profound knowledge of programming syntax and log schemas becomes a necessity for creating these detection rules [8]. This results in a massive operational bottleneck. Tier 1 analysts are typically juniors who lack the deep understanding and years of experience required to identify malicious payloads from random raw JSON logs and immediately create custom-tailored rules to block them. Furthermore, these junior analysts often lack the contextual experience needed to accurately map modern attack patterns to known Advanced Persistent Threat (APT) groups using the MITRE ATT&CK framework [9]. Because they lack this specific expertise, they are forced to escalate almost every suspicious alert to their seniors. Such recurrent escalation clogs the pipeline for senior analysts, who end up performing basic triage instead of actively hunting for advanced threats and critical cases.

To overcome this knowledge gap and decipher complicated logs, many analysts have started resorting to general-purpose Large Language Models (LLMs). Conversely, corporate security telemetry is highly sensitive. It is common for logs to regularly reveal internal IP addresses, server names, employee emails, and sometimes even plaintext passwords that were accidentally typed into the wrong login field. Copying and pasting this type of raw

telemetry into a publicly available, cloud-based AI model is a major breach of data privacy laws and corporate compliance standards [10].

Therefore, the shortcomings of current technologies exhibit a tripartite problem:

1. **Manual Dependency**: The process of extracting threat data and writing defensive rules takes too long and requires too much specialized syntax knowledge.
2. **The Knowledge Gap**: Junior analysts cannot easily map raw log data to complex threat frameworks without constant senior intervention.
3. **Data Privacy Risks**: Utilizing public, cloud-based AI to analyse sensitive network logs creates unacceptable data leakage vulnerabilities.

1.4 Proposed Approach

A privacy preserving framework that can automatically extract data and perform the threat attribution and incident reporting has to be implemented so that the internal corporate data does not end up on the public internet. This is required in order to resolve the pressing challenges listed before.

The Agentic Multi-Tenant Managed Detection and Response (MDR) framework has been introduced in this project in order to accomplish this. The aim we here is not completely replacing our human security analysts as this would just cause such severe ethical and operational risks over time. It has been specifically architectured to function as an intelligent assistant that is operating at high speed. The fundamental idea is to deploy a swarm of decentralized and specialised AI agents over the enterprise tools. These will operate within a secure containerized microservice environment instead of having a homogenous inflexible software model deployed like the other cases.

The framework is able to ingest all the security logs in large volume rapidly with the use of the automated zero trust API gateway that we implemented. All the Personally Identifiable Information (PII) like the names of the employees, their emails and the internal hostnames are all mathematically scrubbed out of the payload by masking with the help of Microsoft Presidio SDK before it is being passed onto any form of AI tool [11]. When this sanitation step is completed, the routing logic transfers this safe data to the appropriate sub-agents, that is working together to hunt any external threat intelligence sources or tools, mapping

behaviours to the MITRE ATT&CK framework and also autonomously draft detection rules that are ready to use.

Hallucination of severity scores leading to faulty values based on just the semantic patterns is a major vulnerability faced while using generative AI in cybersecurity and in order to totally eradicate this, we made the project to utilise the Isolation Forest machine learning algorithm to mathematically score the anomaly detected before the LLM reaches a conclusion [12]. The proposed approach ensures that always very accurate intel is received by the human analyst. By strictly making the LLM to ground its findings to the hard statistical mathematics.

1.5 Objectives

The main goal of this project is to remove the manual obstacles that currently slow down the alert triage workflow. A system that acts as a smart operational assistant for cyber analysts has been created, as opposed to attempting to replace human analysts in the field. All routine, manually exhaustive data-gathering tasks will be handed over to a swarm of automated agents. This ensures that cybersecurity staff do not waste their time on false positives or less critical cases, and instead focus entirely on making high-level security decisions and catching critical threats before they go unnoticed among the heavy alert noise. This is a complex project working towards a solution that is well-suited for a live corporate setting while also providing the maximum learning experience for this internship. This project focuses on six specific core objectives:

- **Mastering Foundational Cybersecurity and Enterprise Architecture:** Before engineering an automated defense system, it is necessary to get a thorough understanding of the environment that requires protection. Therefore, the first objective is a comprehensive study of fundamental cybersecurity concepts, including network protocols, firewall configurations, and the operational mechanics of SIEM and SOAR platforms. Acquiring this in-depth domain knowledge guarantees that the developed AI framework will align perfectly with real corporate network realities.

- **Acquire Hands-On Experience in MDR Engineering and Offensive VAPT**: Building a resilient defensive platform requires knowledge of both the attacker's and the defender's mindsets. This objective is about experiencing SOC procedures through

practical, hands-on experience and understanding Managed Detection and Response (MDR) software engineering. Furthermore, conducting active Vulnerability Assessment and Penetration Testing (VAPT) helps reveal exactly how malicious parties generate the network telemetry that the AI swarm is designed to detect.

- **Automating Alert Triage to Reduce Analyst Cognitive Load:** Security teams spend a huge amount of time every week manually cross-referencing IP addresses and searching for vulnerability signatures. The objective here is to eliminate this swivel chair analysis concept by building a system that automatically ingests raw security logs and converts them into readable summaries. Using language models as cognitive support tools is highly effective in minimizing the mental exhaustion that analysts face during long shifts [4].

- **Accelerate Incident Response Times Using Specialized Agents:** Relying on a single generalized AI chatbot is not only inefficient but also prone to causing logical errors. Therefore, the next objective is to create a decentralized swarm of specialized AI agents. By splitting the workload, different modules will perform specific tasks, such as having one agent extract network indicators while another simultaneously drafts detection queries. This enables the system to process threats much more rapidly. Utilizing smaller, highly targeted security models for specific queries is significantly more efficient for SOC workflows than relying on massive, generalized systems [13], [14].

- **Bridging the Knowledge Gap for Junior Analysts:** Writing highly complex detection rules requires years of programming syntax experience [8]. The objective here is to empower junior team members by providing them with powerful tools capable of automating expert context. The framework is designed to automatically map raw attack logs to the MITRE ATT&CK framework and instantly generate step-by-step remediation playbooks.

- **Enforce Strict Data Privacy and Access Control:** The logs generated by a corporate network are highly confidential, and forwarding this telemetry to any public cloud AI provider introduces severe privacy vulnerabilities [10]. The final objective is to utilize localized microservices to sanitize all logs and guarantee zero data leakage. Furthermore, the framework will implement a strict Role-Based Access Control (RBAC) matrix to ensure that analysts can only access the specific tools and AI agents permitted by their

exact job title [15], [16].

1.6 SDGs

The creation of the Agentic MDR Framework actively contributes to the United Nations Sustainable Development Goals (SDGs), particularly by modernizing digital infrastructure and promoting secure, sustainable work environments [17]. As illustrated in Figure 1.1, this project is part of a broader strategy that intends to secure the continued development of global technology while simultaneously addressing operational inefficiencies within the cybersecurity sector. This framework directly supports three core United Nations Sustainable Development Goals listed below:

Figure 1.1 United Nations Sustainable Development Goals (SDGs) [17]

- **SDG 8 (Decent Work and Economic Growth):** The framework provides a concrete way to help SOC analysts by mitigating extreme alert fatigue and delivering automated solutions for their repetitive, manual triage tasks. This significantly improves the mental well-being and daily working conditions of the security analysts.

- **SDG 9 (Industry, Innovation and Infrastructure):** Engineering a resilient, AI-driven automation pipeline fosters technological innovation and provides a direct way to

upgrade outdated enterprise security infrastructures. This allows enterprises to safely handle the massive modern data volumes that are characteristic of current times.

- **SDG 16 (Peace, Justice and Strong Institutions):** Equipping organizations with advanced, privacy-preserving defensive tools actively combats cybercrime more effectively. This fosters stronger digital institutions that are entrusted with protecting confidential user data from malicious actors.

1.7 Overview of Internship Report

This report details the complete Software Development Life Cycle (SDLC) of the Agentic MDR Framework, organized into nine distinct chapters:

- **Chapter 1: Introduction** establishes the background of SOC operations, the crisis of alert fatigue, and the core objectives of the proposed AI framework.
- **Chapter 2: Literature Review** examines current academic research regarding LLM integration in cybersecurity and data privacy mechanisms.
- **Chapter 3: Methodology** outlines the engineering approach, focusing specifically on Chain-of-Thought reasoning and mathematical anomaly detection.
- **Chapter 4: Project Management** details the internship timeline, technical risk analysis, and budgetary requirements.
- **Chapter 5: Analysis and Design** define the system requirements, architectural block diagrams, and relational domain modeling.
- **Chapter 6: Hardware, Software, and Simulation** specify the technological stack and the methodologies used to simulate live cyber-attacks.
- **Chapter 7: Evaluation and Results** analyse the framework's performance and accuracy across various MITRE ATT&CK threat scenarios.
- **Chapter 8: Social, Legal, Ethical, Sustainability and Safety Aspects** evaluate the regulatory compliance and ethical boundaries of deploying autonomous AI in network defence.

- **Chapter 9: Conclusion** summarizes the final project outcomes and outlines the future scope for platform scalability.

Chapter 2

LITERATURE REVIEW

The field of cybersecurity is constantly evolving, and with this evolution, the interest of the industry has shifted to the use of Generative Artificial Intelligence and Machine Learning based solutions. This chapter divides recent research into different technology concepts: the application of Large Language Models (LLMs) in security operations, the automated generation of detection rules, the reasoning done by autonomous AI agents, and stringent data privacy requirements.

Lochner and Keplinger (2025) carried out an empirical 10 month longitudinal study of live SOC analysts. They found that professionals regarded Large Language Models strictly as a cognitive support tool to help explain confusing logs, rather than considering them capable of making decisions autonomously. The major gap highlighted here is that if human analysts rely too much on AI only for cognitive support, letting fully autonomous systems work independently to block IP addresses or shut down servers would pose a severe operational risk. We solved this problem by proposing an advanced Human-In-The-Loop (HITL) system where our 7 Agent Swarm only performs automated triage, threat hunting, and report generation, whereas the human analyst always retains the final decision on active network remediation [4].

Wang et al. (2025) proposed RulePilot, an LLM powered agent that can take a plain English description of cyberattacks and turn them into executable SIEM rules using the Chain of Thought (CoT) reasoning concept. While the system is very effective at writing accurate syntax, the main problem with RulePilot is that it functions as a separate tool which needs a human analyst to manually identify the threat and type it out into the application, which increases the workload. Our work, however, overcomes this manual dependency. We have embedded CoT reasoning into a completely automated pipeline, whereby our Detection Agent automatically receives verified threat information straight from the webhook ingestion layer, generating rules instantly without needing a human middleman to explain the attack [8].

Muzammil et al. (2025) questioned the idea that massive cloud models are necessary for security operations by demonstrating that fine-tuned Small Language Models could convert human instructions into structured query logic with incredibly high accuracy and at a fraction of the computational cost [13]. The limitation of their work lies in their narrow concentration only on stand-alone query translation instead of the continuous incident response lifecycle. We solve this problem by utilizing optimized routing libraries [18] to dynamically allocate specific lightweight log parsing tasks to smaller models, while reserving heavy enterprise models strictly for complex, multi stage threat correlation.

Chong et al. (2025) addressed the enormous data privacy hazards of web based LLMs by developing Casper, a browser extension that utilizes local Named Entity Recognition (NER) to intercept and redact sensitive corporate telemetry before it goes to public cloud servers [10]. The major loophole of this client side strategy is that browser extensions can be easily circumvented, uninstalled, or disabled by an end user. We defeat this security vulnerability by embedding feature preserving redaction logic directly into the backend infrastructure. By utilizing the Presidio SDK [11] at the API gateway level, we aggressively clean incoming telemetry before the raw logs are ever delivered to the intelligence routing layer.

Dai et al. (2025) analyzed how automated attack investigations could be carried out, finding that threat knowledge augmented into LLMs is extremely valuable for the initial triage of complex network alerts and allows security teams to handle larger data volumes. However, the theoretical models proposed in such research often lack the practical architectural deployment needed for live, multi-tenant enterprise environments. We fill this void by translating these automated log analysis theories into a functional, containerized microservice setup. We engineered dedicated Query and Intel agents strictly to execute these rapid threat assessment tasks via a secure API infrastructure [6].

Male (2025) describes the Role Based Access Control (RBAC) methodology, demonstrating how assigning organizational role based permissions is the best way to ensure compliance with the least privilege access methodology within distributed web platforms. The weakness in the context of cybersecurity AI is that if autonomous agents have the ability to write firewall rules, unrestricted access by a junior intern creates a massive internal security liability. We address this risk by directly applying Male's concepts to build a strict RBAC matrix at the persona level. Utilizing JSON Web Tokens (JWT), we ensure that only authorized, senior level users are permitted to access our API gateway and trigger higher

level remediation activities [15].

Ismail et al. (2025) explored robust security orchestration through hyper automation, arguing that agentic artificial intelligence can autonomously execute the entire incident response lifecycle. However, the primary drawback of this fully automated approach is that it frequently ignores the necessity for transparent reasoning. If an AI acts as a black box, analysts cannot verify why a specific response was triggered, which breaks down the core concept of transparency. We resolve this gap by enforcing strict conversational memory within our Chat Sessions database. This allows the human analyst to actively review the AI swarm's step by step reasoning and intervene if the hyper automation logic deviates from corporate policy [5].

Vitla (2025) investigated how RBAC influences the comprehensive cybersecurity policies of organizations. The research highlighted that while RBAC policies are excellent in theory, they frequently fail in practice because they remain static, human enforced documents that are easily circumvented during emergency incident response scenarios. Our framework brings a novel solution to this flaw by hardcoding the RBAC policy directly into both the API routing layer and the LLM system prompts. If an unauthorized user attempts an action, the system does not simply log a policy violation; it cryptographically drops the network request, mathematically enforcing the policy [16].

Krishnan (2025) advanced the concept of multi agent systems by introducing the Model Context Protocol (MCP), a standardized architecture that allows AI models to utilize external tools and APIs seamlessly. While highly innovative, general MCP implementations lack the rigorous security constraints required for enterprise threat intelligence. We adapted this technology specifically for the SOC environment by engineering custom, security focused MCP connectors that strictly limit the AI to verified threat databases like Google OSV and VirusTotal, preventing the swarm from arbitrarily executing unverified code from the public internet [19].

Fumero et al. (2025) investigated autonomous blue team defenses by developing CyberSleuth, an LLM driven forensic agent designed to investigate web attacks by parsing complex network traces. The primary limitation of this research is its highly reactive nature. The system is engineered to operate almost exclusively on post incident forensic data, which severely limits its ability to intervene and stop an ongoing breach. We overcome this

operational delay by connecting our API gateway directly to live SIEM webhooks. This allows our AI swarm to process attacks and generate deployable blocking rules seconds after the initial alert fires, transforming the AI from a forensic tool into a proactive defense mechanism [20].

Xu et al. (2025) surveyed the rising trend of LLM based autonomous cyberattacks and introduced the term Cyber Threat Inflation to describe how offensive AI reduces the expertise and time required for hackers to launch complex campaigns. The operational gap identified is that traditional, human operated defensive tools are fundamentally too slow to respond to machine speed attacks. Our project directly addresses this imbalance. By deploying a decentralized defensive agent swarm capable of executing log extraction, threat correlation, and rule generation simultaneously, we empower the defending organization to respond to automated attacks at machine speed [1].

Sharma (2025) provided a comprehensive risk analysis regarding the security implications of cloud migration, highlighting the danger of sensitive data leakage in multi-tenant environments. The gap in applying generative AI to these environments is that an LLM might accidentally cross contaminate memory, utilizing Tenant A's proprietary network logs to answer a query for Tenant B. We resolve this massive security liability by implementing strict domain modeling. Every log ingested is tagged with a unique tenant ID at the PostgreSQL database level, and the API gateway cryptographically restricts the LLM from querying or accessing data outside the authenticated user's specific tenant scope [21].

Garg et al. (2023) conducted a comprehensive review on transforming security and privacy utilizing Natural Language Processing (NLP). The drawback noted in using generalized NLP for privacy protection is that traditional models struggle to accurately parse the deeply nested, highly structured JSON schemas typical of enterprise security telemetry. We bring a tailored solution to this gap by implementing a custom sanitization middleware pipeline. This utilizes the en_core_web_sm spaCy model specifically configured to navigate and strip Unified Data Model (UDM) formatted logs, ensuring accurate redaction without breaking the JSON structure [22].

Metin et al. (2025) addressed the unique vulnerabilities introduced by autonomous systems, proposing guidelines for securing agentic AI strictly aligned with the NIST Cybersecurity Framework 2.0. The limitation of their research is that it provides high level theoretical

guidelines rather than concrete technical enforcement mechanisms against novel threats like prompt injection. We translated these theoretical NIST guidelines into tangible code by engineering a custom tokenization based prompt guard. This linguistic firewall actively scans incoming telemetry against a semantic denylist, physically preventing attackers from manipulating the AI swarm's logic [23].

Mitra et al. (2025) described FALCON, an agentic framework that mines Cyber Threat Intelligence (CTI) feeds to autonomously generate Intrusion Detection System (IDS) rules. While highly accurate, the framework relies entirely on external, third party intelligence feeds rather than internal, localized anomalies. Our framework improves upon this by taking a hybrid approach. Our Detection Agent generates highly specific YARA rules based directly on the localized, internal EDR telemetry ingested from the organization's own network, ensuring the defense is specifically tailored to the active attack rather than generic global signatures [24].

In summary, an extensive analysis of contemporary academic literature reveals a critical paradigm shift in enterprise cybersecurity strategies. Manual, human driven log triage is no longer viable against automated, machine speed cyber threats. While researchers have successfully proven that Large Language Models can write detection rules, parse complex forensic telemetry, and translate queries, the current academic landscape is heavily fragmented. Most proposed solutions operate as isolated, theoretical tools that lack the rigorous data privacy, multi-tenant isolation, and Role Based Access Controls required for deployment in live, highly regulated corporate environments. Furthermore, a recurring vulnerability across existing AI research is the risk of generative hallucination and prompt manipulation by advanced threat actors. This project addresses these widespread industry gaps by unifying these isolated concepts into a single, cohesive architecture. By combining the natural language capabilities of a specialized 7 Agent Swarm with the deterministic mathematical grounding of unsupervised anomaly detection algorithms, and enveloping the entire system in localized, privacy preserving microservices, the Agentic MDR Framework transforms theoretical AI research into a highly secure, enterprise ready reality.

Chapter 3

METHODOLOGY

Integrating generative AI into security projects tends to break traditional software engineering methodologies because large language models are known for introducing unpredictable outputs. The Agentic MDR system uses a specialized Agile Software Development Life Cycle (SDLC) that blends deterministic full stack security engineering with non-deterministic machine learning integration. This approach works under the principle of Constrained Intelligence, which limits how the AI behaves. It is divided into three highly regulated modules here: Data Ingestion and Normalization, Agent Swarm Routing, and Mathematical Grounding. Each part operates under strict rules to prevent any uncontrolled behavior. The system prioritizes stability over innovation when handling the AI responses.

3.1 Data Ingestion and Normalization Methodology

A cybersecurity AI would be mathematically useless without the availability of reliable, structured, and sanitized telemetry. If we feed it unstructured or corrupted logs, it will inevitably generate false positives. That is the reason why the first phase of the methodology focuses entirely on establishing cryptographically secure data ingestion pipelines.

The architecture of high throughput RESTful API endpoints required using the FastAPI framework. FastAPI was deliberately chosen over traditional frameworks like Django because of its native support for asynchronous (ASGI) routing. In a live enterprise environment, a cyberattack can generate thousands of logs per second. An asynchronous webhook ensures that the ingestion gateway does not block or drop concurrent incoming HTTP POST requests from our external enterprise platforms.

As shown in Figure 3.1 below, the system aggregates host level telemetry from Endpoint Detection and Response (EDR) agents alongside broad network and identity logs from the Security Information and Event Management (SIEM) platform.

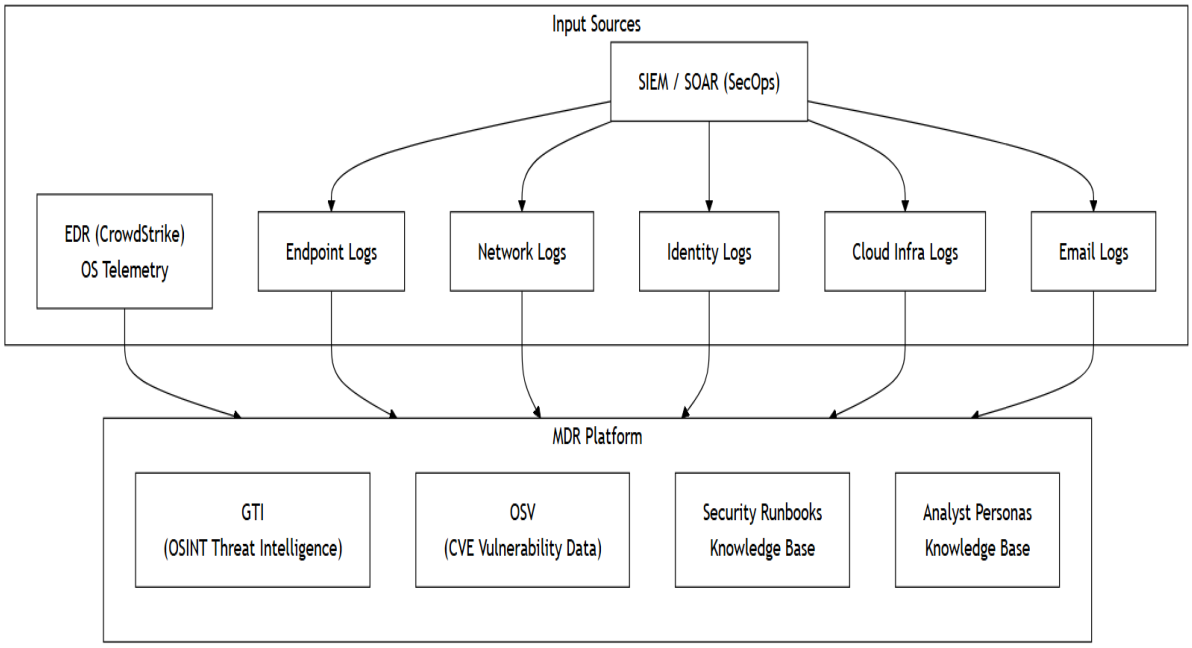

Figure 3.1 Data Ingestion and Intelligence Integration Architecture

We know a significant methodological challenge arises because different security vendors format their data very differently. When CrowdStrike Falcon generates a JSON schema, it is completely different from what we get from Google SecOps. We introduced a strict Data Normalization Layer to solve this problem. Here, the backend parses any disparate JSON payload that comes in and maps it to a UDM schema before it is stored. This ensures that the AI agents can always find the source IP address, destination port, and timestamp in the exact same JSON key fields, no matter where the log came from.

Also, before this normalized data gets stored, it is sanitized via a locally hosted Sanitization Middleware. This layer utilizes the Presidio SDK to mathematically scan the text for pattern matches and remove any Personally Identifiable Information (PII) [11]. Absolute compliance with data privacy regulations is maintained so that the intelligence layer will never see any sensitive data like employee details or internal hostnames [10].

Finally, this clean, normalized data is committed to a relational PostgreSQL database. By writing the data to a hard storage layer before any AI processing occurs, a cryptographically secure audit trail is maintained. The maintenance of these audit logs is a mandatory requirement for digital forensics, ensuring that no volatile evidence is lost even if the cloud intelligence API experiences a timeout or network failure [6].

3.2 AI Routing and Agent Swarm Engineering

One common mistake I have seen in modern AI security tools is the use of the same monolithic Large Language Model (LLM) to perform all tasks, from reading logs to writing rules. This approach overwhelms the context window of the model and frequently leads to hallucinations. That is why we introduced the second phase of the methodology to enforce a decentralized architecture with a swarm of highly specific, isolated agents utilizing advanced Model Context Protocol (MCP) paradigms [19].

I am not using common conversational prompting techniques, but instead enforcing strict Chain of Thought (CoT) reasoning paired with a temperature setting of 0.0 to force deterministic, highly repeatable outputs [8]. As depicted in the below Figure 3.2, the swarm is centrally managed by the Orchestrator Agent, which acts as the cognitive manager. When a new alert is registered in the PostgreSQL database, the Orchestrator evaluates the initial context and delegates highly specific micro tasks to the sub agents via the LiteLLM routing framework [18].

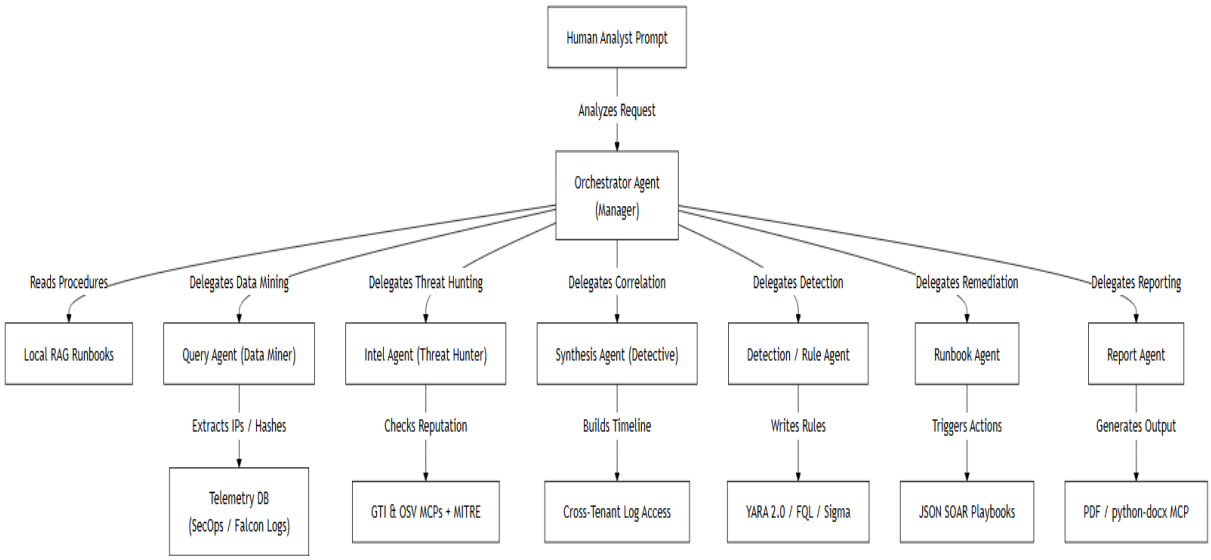

Figure 3.2 Workflow of the Multi-Agent MDR System

Hardcoded system prompts physically restrict the agents from performing tasks outside their designated operational lane in our methodology:

- **The Query Agent:** Engineered solely for the extraction of immutable artifacts like IP addresses, domains, and hashes from the JSON logs. It is mathematically penalized by

the routing logic if it attempts to guess the attacker's motivation.

- **The Intel Agent:** Functions strictly as an API caller. It takes the parsed artifacts and queries external OSINT intelligence databases, ensuring the AI relies on verifiable global threat data rather than its own pre trained, potentially outdated knowledge weights [25].

- **The Detection Agent:** Engineered exclusively to generate syntax. It takes the verified threat narrative and translates it into deployable YARA-L or FQL rules.

By isolating the reasoning paths, this swarm methodology ensures that if the Intel Agent fails to find a malicious hash on VirusTotal, the Detection Agent is automatically blocked from writing a rule. This not only drastically reduces the framework's false positive rate but also prevents the AI from taking any unverified, autonomous actions [20].

3.3 Mathematical Grounding Methodology

The final phase of the methodology required building a deterministic validation layer to anchor the generative AI. A known vulnerability in utilizing LLMs for security triage is that they are designed to predict the next most logical word, not to calculate statistical probabilities. Consequently, they are prone to hallucinating dangerously high or low threat severity scores simply because the prompt contained alarming keywords. I wanted to see what would happen if the system was strictly dedicated to identifying anomalies based on math without making guesses.

The answer is the Isolation Forest, a purely mathematical machine learning algorithm that acts as the final arbiter of truth [12]. It is built using the Scikit-learn library [26] and finds anomalous behavior without massive labeled datasets. Unlike traditional models that try to profile normal network traffic, this methodology assumes that cyber threats are anomalous because they are few and different. The algorithm works by randomly selecting a feature from the ingested UDM logs, such as the frequency of failed login attempts, and randomly selecting a split value. Because anomalies like an SSH brute force attack are statistically rare and vastly different from normal baseline traffic, they require significantly fewer splits to be isolated in the mathematical tree structure.

The methodology calculates the anomaly score based on this path length. If the path length

is exceptionally short, the algorithm flags the event as highly anomalous. Before the AI Report Agent is allowed to draft the final incident response document, the raw numerical network data is passed through this algorithm. The framework vectorizes the log metrics, processes them through the Isolation Forest, and restricts the output to a rigid, binary classification of either High Confidence or Low Confidence. By forcing the generative AI to append this strict, mathematically calculated confidence score to its final dashboard output, the methodology ensures that human security analyst is never relying purely on the linguistic assumptions of LLM. The final recommendation presented to the SOC team is therefore a perfect synthesis of contextual natural language processing and hard statistical mathematics. Confidence is transformed from an assumption into a quantifiable metric with this approach.

Chapter 4

PROJECT MANAGEMENT

Integrating generative AI into a business security environment creates a unique set of project management challenges. Older project management methodologies for software development projects often have difficulty dealing with the non-deterministic, black box behavior of Large Language Models and the stringent data privacy regulations that apply to Security Operations Centers (SOCs) [6]. That is why we necessitated an extremely disciplined Agile Software Development Life Cycle (SDLC) paired with aggressive risk management strategies.

4.1 Internship Timeline

This project was developed and refined through a comprehensive 14-week internship program. The workflow transitioned sequentially from a fundamental knowledge of cybersecurity processes to complex AI engineering and eventually to offensive vulnerability assessments. The major milestones I got to achieve here are broken down below:

- **Weeks 1 - 2: Core Cybersecurity and SOC Fundamentals:** It started with a focus on understanding enterprise network architecture and all the basics. This included me studying the OSI Model, TCP/IP protocols, firewall configurations, and the differences between Intrusion Detection Systems (IDS) and Intrusion Prevention Systems (IPS). The training then moved to SOC operations, focusing on the alert triage lifecycle and the operational mechanics of SIEM and SOAR platforms as I got to be a part of the SOC [3].

- **Weeks 3 - 4: Live SOC Triage and Log Analysis:** Active log monitoring was held in a live SOC environment. I got to perform hands-on analysis of raw security telemetry, validating alerts, and understanding the practical pain points that analysts face during daily triage operations in this alert-crowded SOC [4].

- **Weeks 5 - 6: Project Onboarding and Threat Intelligence Research:** Then I proceeded to the engineering team working on the AI MDR framework and conducted very deep research on all the existing threat detection technologies. This included

analyzing MITRE ATT&CK integration methods in some open-source projects [9], studying the OSV (Open Source Vulnerabilities) database [27], and evaluating Microsoft Presidio for corporate compliance and PII redaction [11].

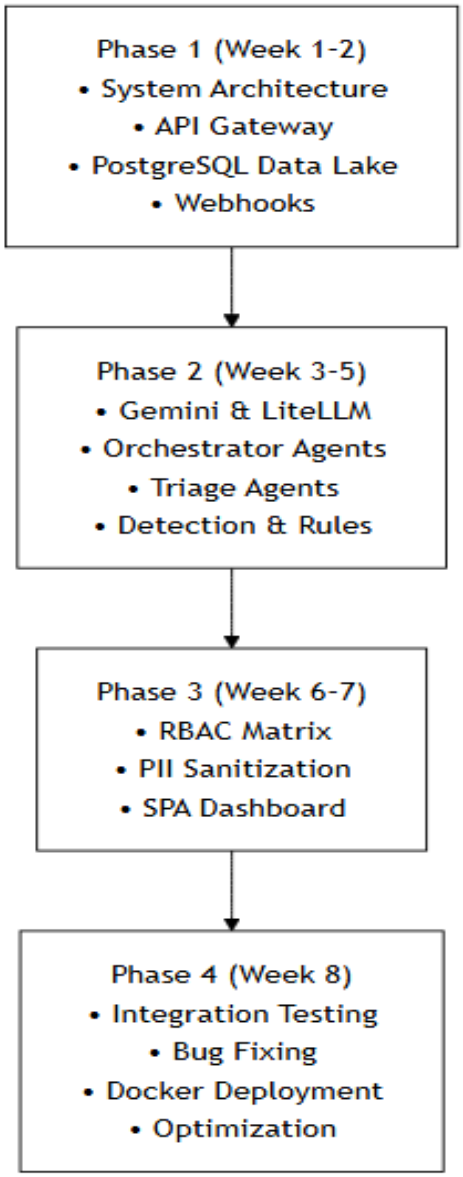

Figure 4.1 Project SDLC Timeline

- **Weeks 7 - 8: Architecture Design and Feature Engineering:** I was able to be a part of the project and draft the core architecture of the multi-agent concept. I provided the engineering team with suggestions on Model Context Protocol (MCP) servers appropriate for specialized tasks like automated diagram generation or PDF and DOCX

report exporting [19].

- **Weeks 9 - 10: Rule Generation and Cloud Security Operations:** I researched automated rule generation capabilities by studying platforms like RulePilot and other LLM-based SOC tools [8]. This helped me understand automated rule generation with LLMs and focus on how telemetry is ingested and parsed by backend API procedures and cloud security operations.

- **Weeks 11 - 12: Vulnerability Assessment and Penetration Testing (VAPT):** Then I got the opportunity to understand the offensive part of security by being in VAPT so I could experience how attackers generate the telemetry our system catches. I trained extensively with Burp Suite Community Edition [28] and completed practical PortSwigger labs to understand reconnaissance techniques, Information Disclosure vulnerabilities, and Cross-Site Scripting (XSS) attacks.

- **Week 13: Final Testing, Documentation and Project Handover:** Finally, I looked into the deployment architecture of the project. Docker containerization [29] was verified, and then I generated the final codebase documentation and completed the formal handover reports.

The complete development timeline and corresponding phases are visible in the flow chart above in Figure 4.1.

4.2 Risk Analysis

Deploying an automated threat triage setup introduces several high-impact errors that might come up. Let us see some very expected ones and how we plan to overcome them:

1. **Risk: API Rate Limiting and Token Exhaustion (High Probability, High Impact)**

    a. **Threat**: For contextual reasoning and threat intelligence, we rely directly on external APIs like Google Gemini and VirusTotal very much [25]. If a massive cyberattack were to happen, thousands of alerts would enter the webhook simultaneously. If the system attempted to query the LLM for every single alert at once, it would lead to an overload of the LLM and cause a platform crash due to the rate-limiting protocols being there.

b. **Mitigation**: Asynchronous queuing will keep requests from piling up. If the system extracts a suspicious IP address, local database caching will surely cut down on daily repeated lookups. Layered redundancy, automated retry logic, and fallback models protect pipeline integrity as all checks are logged in for audits [18].

2. **Risk: Corporate Data Privacy Leakage (Low Probability, Critical Impact)**

   a. **Threat**: Sensitive corporate proprietary data might get accidentally transmitted as we use a cloud-based generative AI service. Sharing any internal network telemetry or employee details will lead to compliance violations, which is a punishable offense [10].

   b. **Mitigation**: We use PII sanitization to avoid this by running the engine locally on the host machine. All sensitive entities are masked from the JSON payload with the Presidio SDK before being sent to the AI routing layer [11]. This guarantees zero data leakage to the public internet.

3. **Risk: AI Hallucinations Triggering False Responses (Medium Probability, High Impact)**

   a. **Threat**: Any complex JSON log being confused by the AI would lead to hallucinations, which is critically threatening for the systems [20]. Analysts blindly trusting these hallucinated severity scores might lead to huge issues, like accidentally quarantining a vital production server.

   b. **Mitigation**: Generative AI cannot assign severity ratings freely; it relies on Constrained Intelligence. It relies on an Isolation Forest machine learning model to measure how odd or statistically different a network event is [12]. These mathematical confidence scores will help the human analyst understand the interpretation in measurable data form, anchoring linguistic assumptions [26].

4. **Risk: Container Orchestration Failures (Low Probability, Medium Impact)**

   a. **Threat**: When the frontend dashboard and the backend API operate in the same environmental container, I see a significant process demand on the AI routing engine might lead to the user interface becoming frozen, which will make the

platform inaccessible for SOC analysts.

b. **Mitigation**: Strict microservice containerization is enforced, where each service executes strictly within its own container. In a bridged virtual network [29], there is a complete separation between the frontend SPA and the FastAPI backend, so the response of the frontend remains unaffected even during heavy computational bottlenecks.

4.3 Internship Operational Requirements

Here the primary objective was to prove that we can make an advanced AI-driven SOC without having to spend millions in licensing fees that would normally add to the cost of a business-level security platform. Leveraging open-source tools safely has proved to decrease the budget requirements, shifting the financial footprint from expensive vendor lock-in to manageable, usage-based cloud infrastructure and API costs.

- **Development and Frameworks (No License Cost):** The whole codebase was made with open-source, enterprise-grade tools like Python and FastAPI for the backend, and Vanilla JavaScript and Tailwind CSS in the frontend. The microservice orchestration was handled entirely by the Docker Engine, so this helped us avoid any commercial hypervisor licenses, completely saving money [29].
- **Cloud Infrastructure and Storage:** The plan was primarily based on building powerful cloud systems on the Google Cloud Platform. As containerized services were deployed with virtual machines optimized for heavy memory routing tasks, structured data was maintained in a centralized Cloud SQL database for PostgreSQL. This was equipped with rapidly responding SSD persistent disks, which are strictly required to handle the high IOPS (Input/Output Operations Per Second) generated by continuous security signals coming in. Because the architecture decoupled the heavy machine learning workloads, expensive dedicated GPU arrays were not required. The Cloud SQL database and standard instances come with an estimated operational cost of $150 to $200 per month, proving enterprise-grade availability under tight constraints.
- **API Token Economy (Varying Cost):** The most critical variable expense is the API token cost for cloud-based Enterprise LLMs, like Google Gemini. Processing raw JSON logs needs huge context windows, and they charge per million input or output tokens. To prevent severe budget overruns, an intelligent proxy router [18] routes high-volume,

low-complexity tasks to cheaper, lightweight models [13]. These models run at about $0.075 per million input tokens. With an average SIEM log needing 2,000 tokens to start, the inference cost per alert drops to around $0.00015. Heavy, expensive reasoning models are exclusively reserved for complex tasks, successfully balancing elite reasoning capabilities with strict financial efficiency.

Chapter 5

ANALYSIS AND DESIGN

The analysis and design stage fills the gap between the theoretical methodology and the actual software engineering needed to construct the Agentic Managed Detection and Response (MDR) Framework. Implementing an enterprise-scale cybersecurity system requires careful design to ensure that the ingestion pipelines will not collapse under the immense pressure of live network traffic. The chapter outlines the architectural specifications, the modular unit designs, the database schemas, and the stringent communication models that govern how the multi-agent swarm securely interacts with enterprise telemetry.

5.1 Requirements

The architectural requirements for the Agentic MDR Framework were strictly divided into functional and non-functional categories to ensure enterprise-grade stability and strict adherence to data privacy laws.

Functional Requirements:

- **High-Volume Asynchronous Ingestion:** The system must feature a highly robust API backend capable of asynchronously ingesting massive volumes of JSON-formatted telemetry. It must maintain stable concurrent connections to external enterprise platforms, specifically Google SecOps [3] and CrowdStrike Falcon [2], without dropping payloads during peak traffic.

- **Automated Natural Language Sanitization:** The architecture must autonomously strip sensitive Personally Identifiable Information (PII), such as employee names, internal MAC addresses, and proprietary domains, using localized natural language processing before any data is routed to the external LLM engine [11].

- **Multi-Agent Orchestration:** The intelligence layer must dynamically route discrete tasks among specialized LLM agents (e.g., Query, Intel, Synthesis). The system must

utilize Chain-of-Thought reasoning to prevent a single model from becoming overwhelmed by excessive context.

- **Automated SOAR Reporting:** The system must autonomously generate downloadable, properly formatted incident response reports (e.g., .docx files). These reports must contain extracted evidence, threat attribution, and remediation steps to satisfy compliance auditing requirements.

Non-Functional Requirements:

- **Sub-Second Latency and High Availability:** Webhook ingestion must operate with near-zero latency. If the network experiences a volumetric Denial of Service (DoS) attack, the ingestion layer must queue the logs efficiently to prevent buffer overflows or data loss.

- **Strict Data Isolation:** The framework must support multi-tenant environments, ensuring that telemetry from one corporate department is cryptographically isolated from another [21].

- **Decoupled Architecture:** The frontend user interface must be completely decoupled from the backend routing logic. This ensures that the analytical dashboard remains highly responsive to the human analyst, even if the backend is processing heavy AI computational workloads.

5.2 Block Diagram

The decoupled frontend, backend, and intelligence layers detailed in Figure 5.1 create an easy framework that works efficiently. The system architecture was designed as a modular microservice environment. The system is separated into specialized blocks: the Data Ingestion Gateway (FastAPI), the Persistence Data Lake (PostgreSQL), the Intelligence Core (LiteLLM router and Google Gemini API), and the Frontend Interface (Vanilla JavaScript Single Page Application). This decoupling ensures high availability.

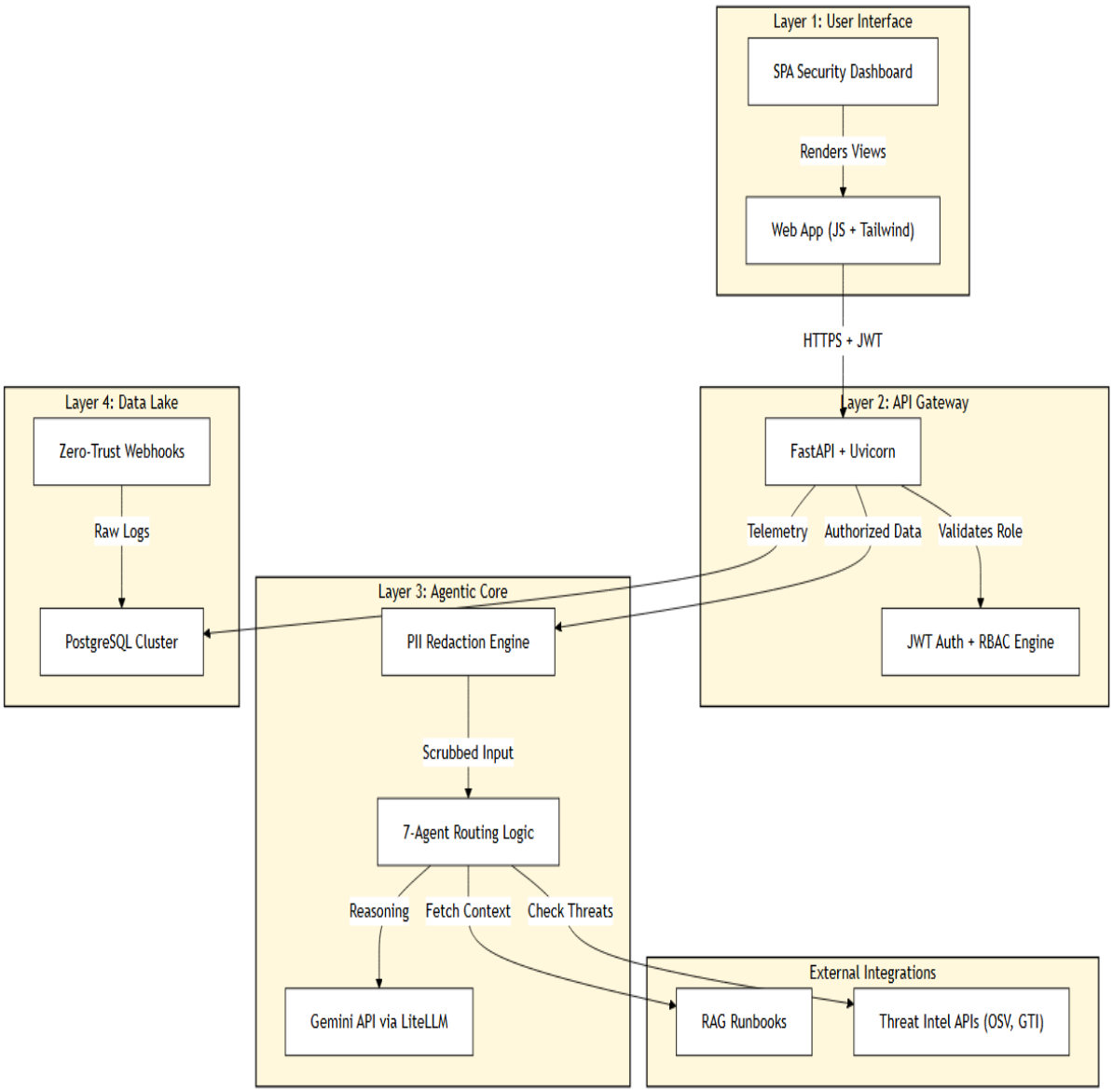

Figure 5.1 High-Level Block Diagram of the Agentic MDR Architecture

5.3 System Flow Chart

Figure 5.2 represents the flow of data through the system to ensure cryptographic integrity and keep a proper audit trail. When an anomaly is detected by the endpoint sensor, the incident is converted into a standardized JSON payload and transmitted to the MDR framework. The sequential lifecycle of a raw log passing through these components is clearly depicted. It is first passed through a PII sanitizer to mask sensitive information. Log metrics are then scored by the Isolation Forest algorithm as planned throughout the project. Finally, the cleaned context is handed over to the 7 Agent Swarm for Natural Language

Processing (NLP), threat hunting, and automated rulemaking.

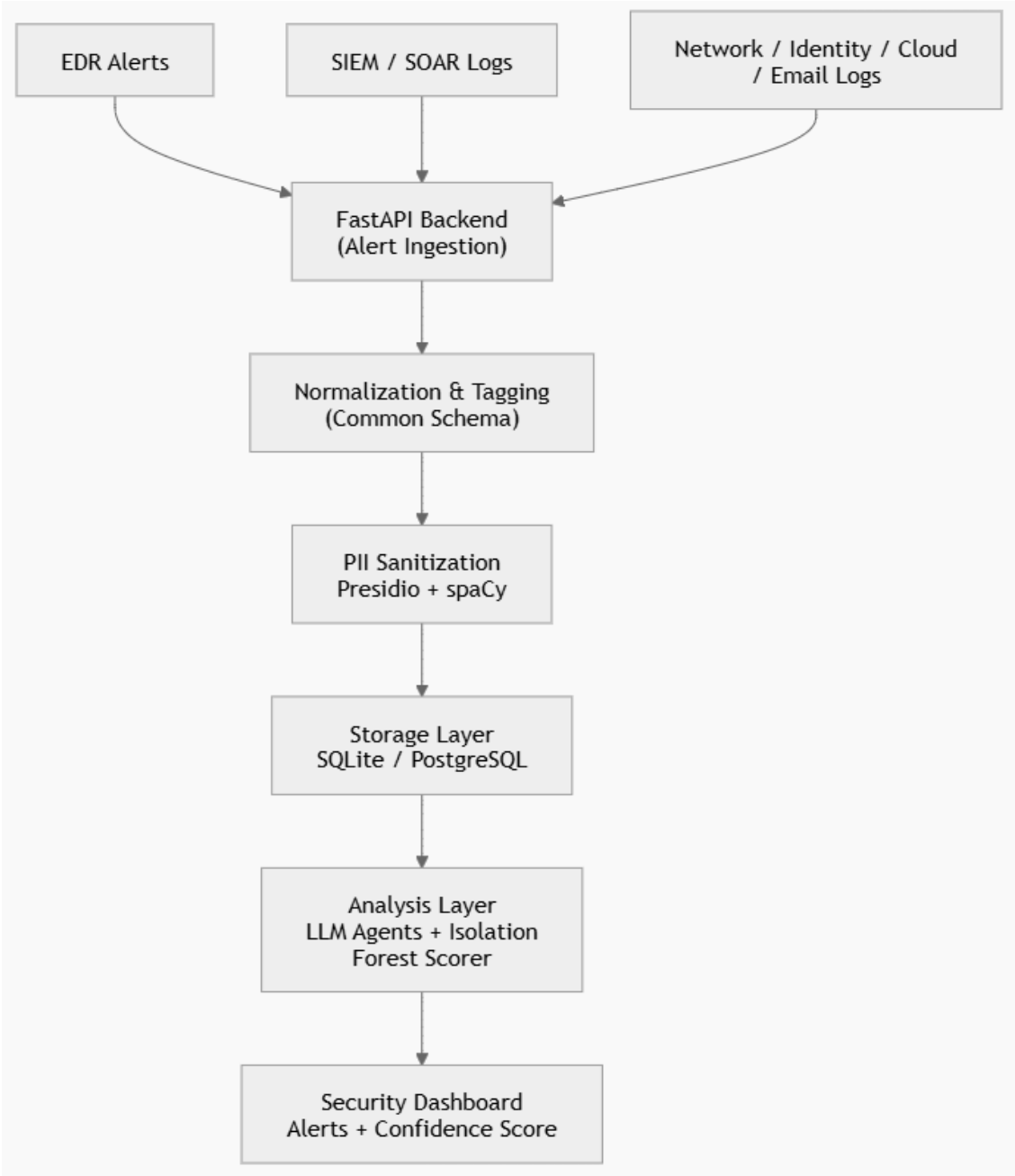

Figure 5.2 Sequential Data Flow and Alert Lifecycle within the Framework

5.4 Choosing Devices

Unlike traditional localized desktop software, this framework operates entirely on cloud native infrastructure utilizing highly optimized virtual hardware and containerized environments.

- **Edge Telemetry Sensors:** The employee laptops have Endpoint Detection and Response (EDR) agents installed locally. These act as the physical sensing devices that basically tell our system about any suspicious or malicious activities happening on that device. It is being continuously monitored for process executions and registry modifications.

- **Cloud Compute Infrastructure:** To prevent processing bottlenecks during concurrent Docker container orchestration, the host environment requires robust virtual machine instances. A minimum of 8 virtual CPU cores (vCPUs on x86_64 architecture) and 16GB of RAM ensures that the FastAPI gateway and the AI routing framework work entirely in active memory. This prevents the host machines from getting stuck or being inaccessible because of slow disk swapping bottlenecks [29].

- **Storage Devices:** Solid State Drive (SSD) block storage is strictly chosen over standard Hard Disk Drives for the database cluster. SSDs are required here to handle the extremely high Input/Output Operations Per Second (IOPS) generated by continuous, high speed SIEM telemetry streams.

5.5 Designing Units

The modules for this project have been designed with a focus on building a system that is less complex, easily understandable, and highly performing. To achieve maximum scalability, fault tolerance, and maintainability, the codebase was designed using a strict separation of concerns following microservice paradigms. Since this has to be deployed in the domain of cybersecurity where extremely sensitive data and processes take place, the security of it all is of immense importance to enterprises and must be looked into properly with the help of these useful AI assisted technologies.

- **The Server/API Unit**: The first part will be our server built using Python 3.10, FastAPI, and a Uvicorn ASGI server. This will function as the very front end of our project and the primary operational gatekeeper. All movement of data or UI concepts will happen through this layer only. It will strictly validate our JSON payloads using Pydantic models to ensure they conform exactly to the UDM schema before any processing occurs. It handles over 20 distinct RESTful endpoints. Webhooks for live alert ingestion from SecOps and Falcon, JWT authorization, session tracking, and SOAR export generation will all be running asynchronously with ASGI to ensure high volume network

telemetry does not block concurrent user requests at the dashboard.

- **The AI Orchestration Unit:** This serves as the cognitive brain layer of the project. I did not want to use one single monolithic AI model to perform all tasks itself, so I introduced a swarm of agents to break the tasks into subtasks and work together towards a single goal. The orchestrator agent dynamically evaluates incoming threat contexts and manages what tasks are taken up by which agents via an intelligent proxy routing layer. It will route the agents appropriately, deploying one for data mining, another for writing detection rules, and another for gathering threat intel. There are 7 decentralized agents in the swarm, each with their own specialized micro tasks to perform using sequential mapping and Chain of Thought (CoT) reasoning to securely aggregate outputs and avoid context window overflows and generative hallucinations [19].

- **The Knowledge Retrieval (RAG) Unit:** This is the foundational knowledge layer. Pre trained general LLMs inherently lack company or topic specific details required to identify what is suspicious and what is normal. They do not have details on how a particular company reacts to certain threats or how the team follows procedural reporting hierarchies. This unit functions as a localized Retrieval Augmented Generation (RAG) pipeline by gathering all the external information we provide it. It uses tokenized relevance scoring to fetch local Markdown files like basic runbooks, personas, and playbooks directly from the internal file system. The AI uses these as context to be aware of its role and duty, grounding it in established operational procedures so it does not hallucinate fake data or make grave mistakes that might cost the business a fortune. The RBAC is properly maintained with the persona specified to it, making our AI highly accurate and compliant with the unique security posture of the organization.

- **The Tools and Integration Unit:** The tool integration unit is of utmost importance. We have used deterministic heuristic logic and Model Context Protocol (MCP) clients here. This helps the client connect securely with external third party infrastructure and gives convenience. We have integrated threat intelligence tools like Google Threat Intelligence (GTI) and the Open Source Vulnerabilities (OSV) database to verify the severity and reputation of the extracted IP addresses and file hashes [27]. Additionally, it utilizes the python docx library to autonomously parse finalized outputs and export the formally structured Incident Response report as a document for compliance auditing.

5.6 Standards

To guarantee that our framework properly and conveniently works within the global cybersecurity infrastructure, we must strictly adhere to established industry standards to ensure it is reliable. In the absence of them, the entire process would not be compatible with other existing security tools.

- **Threat Categorization:** The extracted abnormal behavior and AI generated threat narratives are mapped strictly to the MITRE ATT&CK framework, which is a universal knowledge base for identifying the tactics and techniques used by attackers. Rather than allowing the AI to invent names, we trace back to find vulnerabilities and strictly show the official Tactic and Technique ID codes of the attacks. This standardization assists the output in being universally comprehended by any security professional [9].

- **Data Normalization:** A major disadvantage in SOC operations is the lack of a universal logging format. Every activity must be recorded to be audited as evidence in the future, but incoming logs are based on varying vendor schemas and are not identical. We need to programmatically normalize them to ensure they do not vary in format or information. We unify all the incoming JSON payloads into the UDM format [3] so they can be parsed consistently by the agents, drastically reducing the token load on the LLM context window.

- **Compliance Auditing:** Heavy regulations must be adhered to when dealing with organization level security platforms. The system maintains an immutable audit logging engine that records every single alert received and operation performed. These are recorded in a proper, unalterable format with strict ISO 8601 and SOC timestamps. This cryptographic tracking ensures that all automated actions taken by the AI can be legally audited, guaranteeing compliance with strict enterprise reporting standards.

5.7 Mapping with IoTWF Reference Model Layers

The IoT World Forum (IoTWF) 7 Layer Reference Model was initially designed to fit the Internet of Things, but we can successfully apply it to the distributed edge to cloud architecture of a contemporary SOC as well, as prepared in Figure 5.3.

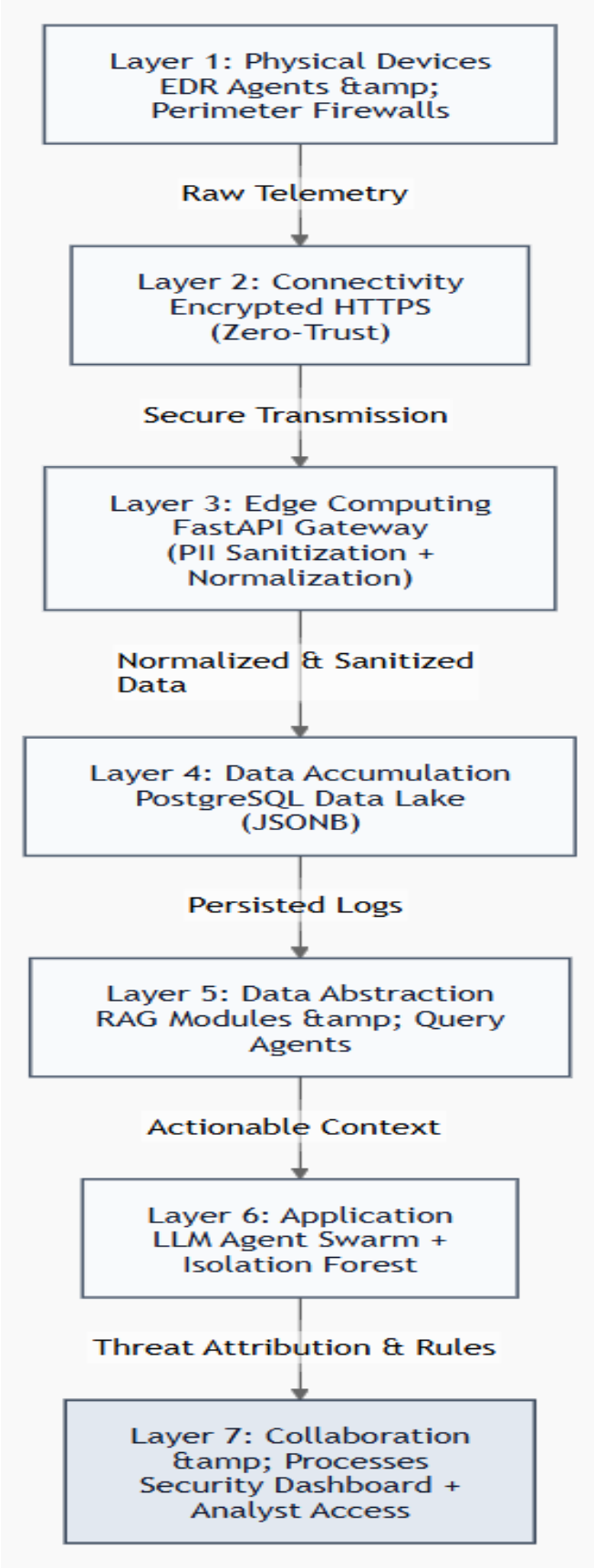

Figure 5.3 Mapping the Agentic MDR Architecture to the IoTWF 7-Layer Reference Model

- **Layer 1 (Physical Devices/Sensors):** This is the physical layer. In a normal IoT project, temperature or motion detectors are attached here. In our cybersecurity MDR framework, this maps directly to the EDR agents, physical firewalls, and other sensors on employee endpoints that continuously sense for system abnormalities [2].

- **Layer 2 (Connectivity):** This represents connectivity. It contains the secure HTTPS zero trust webhooks that provide the JSON telemetry from the physical endpoints to our centralized MDR system via encrypted internet routing.

- **Layer 3 (Edge Computing):** The FastAPI gateway is used to perform limited, low latency edge computing operations such as PII sanitization and UDM normalization before the data passes deeper into the intelligence core. These tasks are done prior to processing with no time wasted.

- **Layer 4 (Data Accumulation):** The localized PostgreSQL data lake. This is where the massive influx of immutable JSON log payloads is securely persisted and indexed in long term storage before the agents actively query the system.

- **Layer 5 (Data Abstraction):** This layer of the model consists of the RAG modules and our Query Agents. They are tasked with filtering and retrieving context from our data lake and other external knowledge files. Raw details like user IDs and IP file hashes extracted out of the data storage are abstracted into actionable information that assists us in acquiring analytical insights.

- **Layer 6 (Application):** This is the core intelligence layer in which all the cognitive processing of input tokens occurs. The 7 agents of the LLM swarm coordinate, collaborate, and perform cross tenant threat attribution based on the workflow delegated to them. Their mathematical anomaly detection is also performed here by the Isolation Forest, ensuring all heavy cognitive and defensive syntax generation operations are handled at this level.

- **Layer 7 (Collaboration & Processes):** This is the human machine interface that analysts will see when viewing the SPA dashboard. Under this layer, they can see the correlated data, review agent activity, connect with the agents to receive suggestions on drafted playbooks, and authorize active remediation protocols and written reports.

5.8 Domain Model Specification

Telemetry information from enterprises that we are ingesting should fall strictly within the boundaries of a highly standardized relational database design built using the Domain Model. Attempting to store any unstructured or corrupted data will inevitably crash applications when seeking information in the AI orchestration layer. Three primary, heavily normalized SQL tables are used to ensure high speed querying and data integrity are maintained when the database is in an active cyberattack state. This offers a schema designed specifically to execute fast read/write capabilities and strict compliance audit tracking.

- **Telemetry Table:** This telemetry table serves as the key data lake of the framework. It stores the raw, immutable format of inbound security alerts by utilizing the rich JSONB data types of PostgreSQL. This offers secure storage of highly flexible and profoundly embedded vendor payloads without necessitating continual schema updates. To enforce the UDM (Unified Data Model), the domain logic dictates that every piece of data written to disk needs to contain a collection of certain top level keys (timestamp, source_ip, destination_ip, event_type, and raw_payload). Crucially, a B-tree index is added to the timestamp field, allowing the AI Query Agent to perform lightning fast temporal searches to find and reassemble attack kill chains from a certain time period very quickly.

- **Chat Sessions Table:** Generative AI models are inherently stateless. There is no memory of past occurrences since they have no built in memory to recall past interactions. Consequently, the temporal history of human to AI investigations will be recorded in the Chat Sessions table to act as a conversational memory tool, tracking the progress of an ongoing event at the moment it happens. This table will contain conversational memory vectors and the contextual state of the LLM window during the interaction. The primary key of this table will be the unique session ID, and the related alert ID will be a foreign key. This ensures the threat hunter can correlate their discussions regarding alert investigations directly with the network data created during their activity.

- **Audit Logs Table:** Because the platform handles sensitive actions, the Audit Logs table functions as an append only ledger where actions performed against the platform are

recorded. This includes, but is not restricted to, user login events, specific AI agent invocations, and the active creation or removal of network defense rules (all of which are regarded as highly sensitive). Therefore, this table must bear cryptographic tracking mapped to a specific user to prevent tampering, providing the immutable security and integrity evidence required for compliance audits.

5.9 Communication Model

Data security is guaranteed through the application of a secure, zero trust model of communication via RESTful APIs. Because security telemetry is highly sensitive by nature, external platforms do not connect to the framework through any other means; they push data exclusively through an HTTPS POST webhook over a TLS 1.3 encrypted connection. Moreover, the microservices interact asynchronously to avoid blocking the processing of requests by another microservice. To ensure the AI Intelligence Layer cannot be accessed or probed directly by the public internet, the API, Database, and UI microservices will only communicate within an isolated and bridged Docker network. The host machine will only open the minimum number of ports necessary to access the UI and the webhook, thereby establishing a secure internal enclave of security [29].

5.10 Cloud / Enterprise Deployment Level

The framework is implemented at an Advanced Enterprise Cloud deployment level (equivalent to Level 6 of IoT as a Multi-Tiered Architecture). Recognizing that monolithic applications tend to crash due to an enormously increased load of data, the system is not a localized desktop program. It is far more cost effective and reliable to control. The system uses a strictly containerized, stateless form of a 2 Tier Primary Application Architecture, orchestrated through the use of Docker Compose:

- **The Backend API Container:** This container houses the FastAPI Core, the NLP application for sanitization of PII, the PostgreSQL Database Connectors, and the proxy routing logic of the LiteLLM agents. These containers are entirely stateless, allowing them to be scaled horizontally in case of a massive growth or spike in the number of alerts.

- **The Frontend UI Container:** This container provides the lightweight Single Page

Application (SPA) dashboard to the end user through a localized web server. By utilizing environment variables (.env files) and persistent volume mounts for the database, the architecture facilitates easy deployment and backup of the system. This also ensures cross compatibility, allowing the same solution to be seamlessly scaled across different cloud providers like GCP, AWS, or Azure.

5.11 Functional View

Functionally, this system operates similarly to a highly automated, multi stage triage pipeline. Human analysts do not have to drag data manually; the data itself pushes through a rigid, functional execution route:

1. **Ingestion:** Telemetry data enters the gateway asynchronously.
2. **Normalization:** The data is instantly converted to the standardized UDM format.
3. **Sanitization:** The NLP engine completely strips and removes PII.
4. **Scoring:** The Isolation Forest is used to mathematically evaluate and test anomalies.
5. **Contextualization:** The AI agents examine the threat narrative and map it strictly with MITRE ATT&CK.
6. **Remediation:** Defensive YARA and FQL rules are autonomously generated.
7. **Presentation:** The human analyst reviews the fully correlated data presented on the finalized dashboard.

5.12 Mapping Deployment Level with Functional View

The containerized Docker deployment has a perfectly isolated nature so that individual functional failures will not lead to the total downfall of the entire system [29]. The frontend container is solely in charge of the Presentation operation, possessing the capability to render views and charts for the user. The backend container, in turn, strictly handles the Ingestion, Analytics, and Security operations. The functional isolation of the AI routing logic from the data accumulation logic assures that a failure or timeout of an external API will never have any impact on the live alert logging.

5.13 Operational View

The operational view shifts dynamically and functions in a fluid manner modified according to who is using the system, controlled by the Role Based Access Control (RBAC) matrix. As a Tier 1 Security Operations Center (SOC) analyst, the operational view is rather straightforward and highly simplified. An analyst logs into the dashboard and is presented with a single priority queue of alerts sorted by the confidence level of the Isolation Forest, eliminating the need to read all the raw JSON logs across multiple different vendor dashboards. From the activity perspective of a SOC Manager or a Risk and Compliance Officer, the operational view is far more focused on generated SOAR playbooks, unalterable audit reports, and strategic MITRE ATT&CK coverage maps.

5.14 Other Design

To achieve maximum security of the internal systems, the architecture includes two major proprietary defense designs:

1. **14 Persona Role Based Access Control (RBAC):** Internal data breaches or system downtime that take place within an organization frequently happen because of the over privileges granted to users who access organizational data in a traditional Security Operations Center (SOC). To ensure this does not occur, we have come up with a highly granular 14 Persona Role Based Access Control (RBAC) matrix fully incorporated into the SOC architecture. Figure 5.4 represents the specific paths of operation, dashboard views, and agent tools that can be used by an authenticated user depending on their persona descriptor. We utilize a mechanism called JSON Web Tokens (JWT) to cryptographically verify, through asymmetric encryption, what organizational role they play on every single API request. Each time an authenticated user transmits information through our API gateways, a JWT is created by the FastAPI backend containing an encrypted description of their assigned persona in the token payload. The API Gateway Middleware intercepts and monitors all requests made to the cloud, cross referencing each token against a strict access control list before authorizing it to proceed with processing. With these cryptographic standards applied, we have mathematically verified the principle of least privilege for each authenticated user or persona in the microservices environment. For example, a Tier 1 Analyst will not be able to do anything with the Query Agent except use it in a read only mode when viewing logs. Conversely, a Tier 3

Incident Responder possesses the required high-level clearance to invoke the Detection Agent and create active cyber defense rules, while a Compliance Manager is strictly restricted to accessing the Report Agent and immutable audit logs [15], [16].

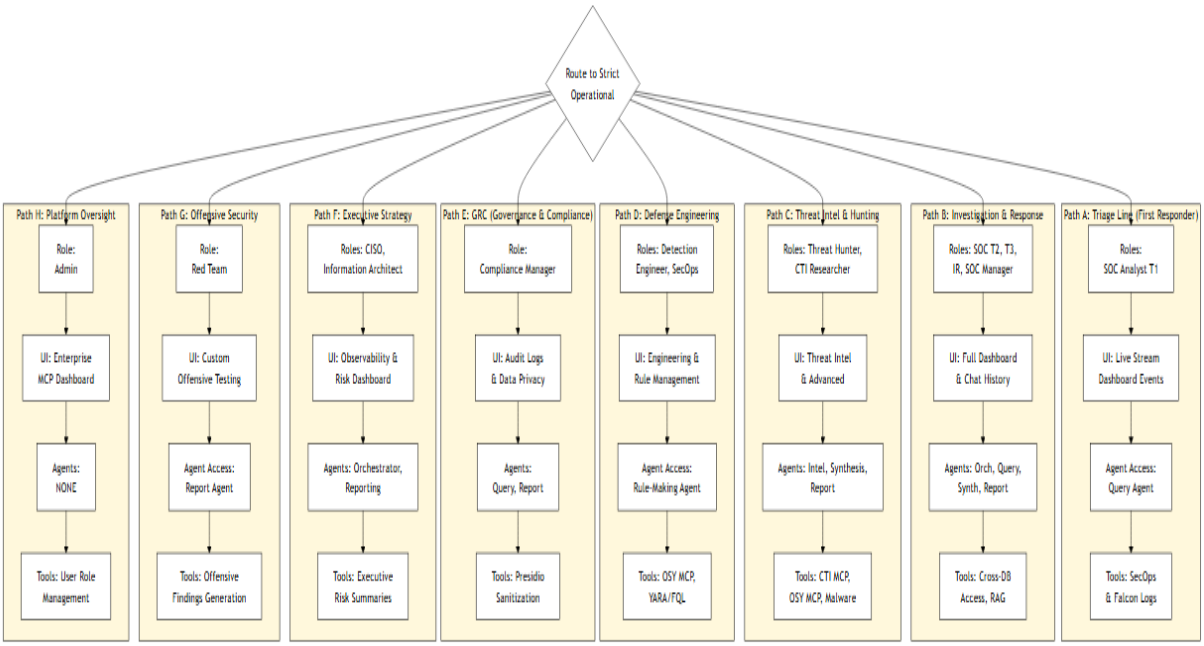

Figure 5.4 Authentication, RBAC Enforcement, and Persona-Specific Operational Paths in the MDR System

2. **Prompt Injection Defence:** As Large Language Models are integrated into automated data ingestion pipelines, they become highly susceptible to indirect prompt injection attacks. A sophisticated adversary can embed malicious natural language instructions directly inside a standard web request, such as a User Agent string or a simulated SQL payload. If the LLM reads this raw telemetry during triage, it may mistakenly execute the attacker's embedded text as a system command. To neutralize this critical vulnerability, the framework implements a custom tokenization-based prompt guard that acts as a linguistic firewall for the AI swarm. Before any log is injected into the LLM context window, this module performs semantic analysis against a constantly updated denylist of adversarial phrases. Furthermore, the orchestration logic strictly isolates the ingested telemetry within rigid delimiter boundaries, explicitly instructing the LLM to treat the enclosed payload purely as structural evidence rather than executable instructions. If the prompt guard detects a high probability of an injection attempt, it aggressively sanitizes the payload and strips the operational verbs, ensuring the integrity of the AI reasoning remains completely uncompromised [1].

Chapter 6

HARDWARE, SOFTWARE AND SIMULATION

Organizations must have a well-designed technology stack to implement an enterprise grade AI platform. The hardware, the specific software frameworks on which you build your microservice architecture, the underlying codebase libraries which form the basis of your framework, and the rigorous methodologies by which you simulate real life cyber threats to safely test the system are all specific technological requirements. This chapter aims at describing all these technologies, the way they interoperate, and the methodologies used to simulate different cyber threats.

6.1 Hardware

In contrast to traditional standalone, monolithic desktop applications, the Agentic MDR Framework is conceived as a cloud native system designed to process thousands of network events asynchronously. The hardware requirements are thus focused heavily on extremely fast memory and storage, instead of traditional localized graphics processing.

- **Compute (CPU & RAM):** The host environment consists of cloud provisioned Compute Engine instances which are required to have a minimum of 8 virtual CPUs (vCPUs on an x86_64 architecture) operational at all times, along with at least 16GB of active RAM. This is a critically important memory limit that will ensure that the FastAPI webhooks and the localized Presidio Natural Language Processing (NLP) engines hosted on the hardware will actually run entirely in active memory. In case of a lack of sufficient RAM, the host environment will resort to disk swapping, severely adding to webhook execution time latency and causing incoming live security alerts to be lost or dropped.

- **Storage Infrastructure (SSD IOPS):** Solid State Drives (SSD) block storage had to be used as the storage medium for the database cluster. Traditional Hard Disk Drives (HDD) suffer from physical seek times that limit the ability to write continuous streams of JSON telemetry, acting as a bottleneck. SSDs offer the required high volumes of

Input/Output Operations Per Second (IOPS) required to write records into the Security Information and Event Management (SIEM) database in real time without locking or blocking access to the database tables.

- **GPU Independence:** The other major architectural choice made was to separate the heavy generative artificial intelligence inference from the local hardware, thereby removing the requirement to create any costly dedicated GPU arrays in the local environment. Since the processing framework utilizes intelligent routing libraries to route complex logical problem solving processes to cloud based Enterprise APIs like Google Gemini, the need for local hardware to execute complex logical problem solving has been eliminated, thereby greatly reducing operational hosting costs for the enterprise.

6.2 Software Development Tools

The data ingestion components, the machine learning algorithms, and the User Interface are seamlessly interoperable due to the carefully curated collection of technologies developed for this stack.

- **Python 3.10 (Core Backend):** Python was chosen as the primary backend language because it is the most popular and universal programming language in the integration of AI and ML. This choice is supported by the fact that it provides native support for all the numerical and NLP library dependencies required to execute this implementation. Utilizing other programming languages like Java or C++ would have required the creation of complicated ML wrappers from scratch to make them usable in our base code, which would have unnecessarily increased the level of technical debt and complexity.

- **FastAPI and Uvicorn:** For the API gateway, FastAPI was used instead of older frameworks such as Django or Flask because it is based on the Asynchronous Server Gateway Interface (ASGI) standard. This enables the simultaneous processing of thousands of concurrent HTTP POST requests without blocking the main execution thread of the application, which is a mandatory requirement when ingesting large amounts of telemetry data coming from multiple third party EDR solutions.

- **PostgreSQL / SQLite Enclave:** A relational database engine was chosen due to its advanced native support for the JSONB data type. Given that various security vendor

products format their alerts differently, it cannot be forced that a hard or rigid fixed SQL schema should be used in the storage of their raw payload information. JSONB provides us with the option to store enormous quantities of unstructured, deeply nested JSON logs while still indexing the timestamp fields, allowing us to query such JSON logs extremely fast with the help of AI solutions.

- **Vanilla JavaScript and Tailwind CSS:** The frontend Single Page Application (SPA) was constructed using Vanilla JavaScript and Tailwind CSS for styling, avoiding the heavy support of JavaScript frameworks like React or Angular. Consequently, the lightweight principle is strictly adhered to in the frontend of the application. Since the amount of AI generated text being fed into browsers is very large, the frontend remains exceptionally lightweight, ensuring the dashboard renders instantly as soon as the browser receives the AI generated text streams.

- **Docker Engine (Containerization):** As illustrated in Figure 6.1, the entire architecture has been established and containerized using Docker Compose [29]. This was implemented to make sure the software will execute identically across development, testing, and production environments. The user interface and APIs were developed in their own distinct virtual environments. This isolated user interface does not have any backend dependencies, preventing conflicts from arising and allowing the API to be restarted without necessarily having any impact on the user interface.

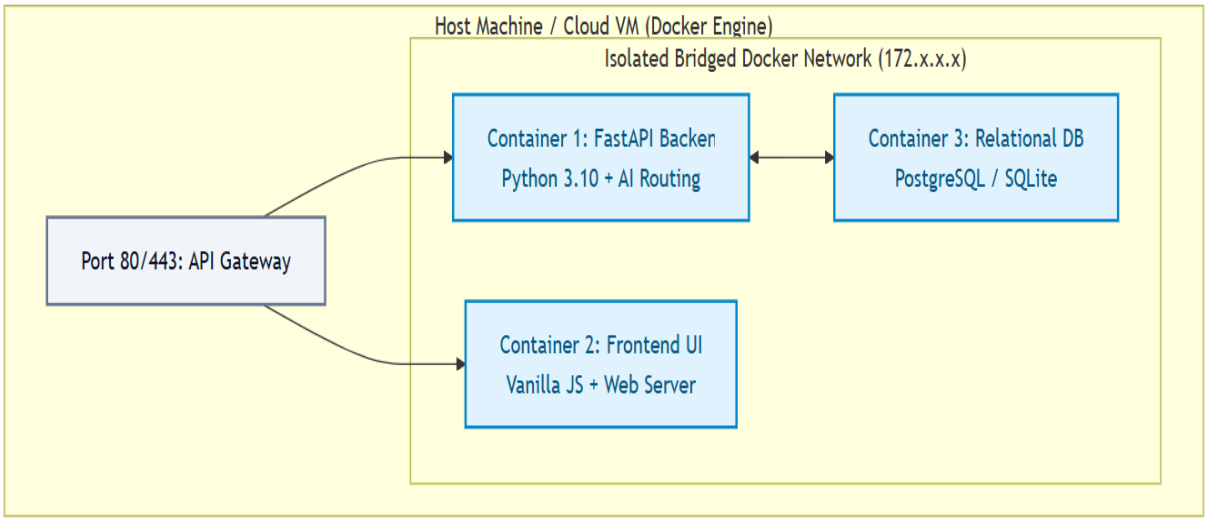

Figure 6.1 Docker Container Architecture of the Agentic MDR Framework

6.3 Software Code

The custom Python codebase software has been engineered with the help of several sophisticated libraries to achieve very specific security capabilities, as detailed in Table 6.1.

Table 6.1 Core Software Libraries and Engineering Functions

Software Library	Primary Framework Function	Engineering Justification
litellm	AI Proxy Routing Engine	Standardizes API calls, allowing the Orchestrator Agent to seamlessly transmit prompts to various LLMs without handling complex raw HTTP requests [18].
presidio-analyser & spacy	PII Sanitization Middleware	Utilizes local Named Entity Recognition (NER) models to mathematically identify and redact confidential corporate data before the AI processes it [11].
scikit-learn	Anomaly Detection Scoring	Powers the IsolationForest module to calculate the statistical deviation of incoming network traffic, mathematically grounding the AI to prevent hallucinations [26].
PyJWT	Cryptographic Authentication	Generates and decodes the JSON Web Tokens utilized by the RBAC matrix, cryptographically ensuring analysts cannot bypass persona permissions.
python-docx	Automated SOAR Reporting	Autonomously parses the AI's Markdown headers to generate formal, formatted .docx incident response files for compliance auditing.

The decision regarding which specific library is selected is not random; they offer a highly interconnected ecosystem which combines advanced machine learning within a rigid enterprise security context. In the Framework, litellm is combined with scikit-learn to effectively couple non deterministic natural language processing with deterministic mathematical scoring. This successfully reduces the frequency of generative AI hallucinations by several orders of magnitude during an active cyberattack. Furthermore,

presidio-analyzer has been incorporated to mask sensitive data, PyJWT to control zero trust access, and python-docx to generate automated documentation. These were introduced to ensure that the entirety of the alert lifecycle, from initial telemetry ingestion to ultimate SOAR report delivery, is rigidly compliant with corporate data privacy laws and ISO/SOC compliance standards. This modular stack of libraries enables the Agentic MDR Framework to operate as a highly secure autonomous pipeline with no dependencies on any heavy, monolithic, or proprietary vendor solutions.

6.4 Simulation

The difficulty of evaluating any cybersecurity platform is that you cannot morally or legally unleash live, destructive payloads into a production corporate network just to generate alerts. This is why a rigorous simulation methodology was required to develop authentic real threat telemetry. The entire step by step sequence of this automated data ingestion and analysis pipeline is shown in Figure 6.2.

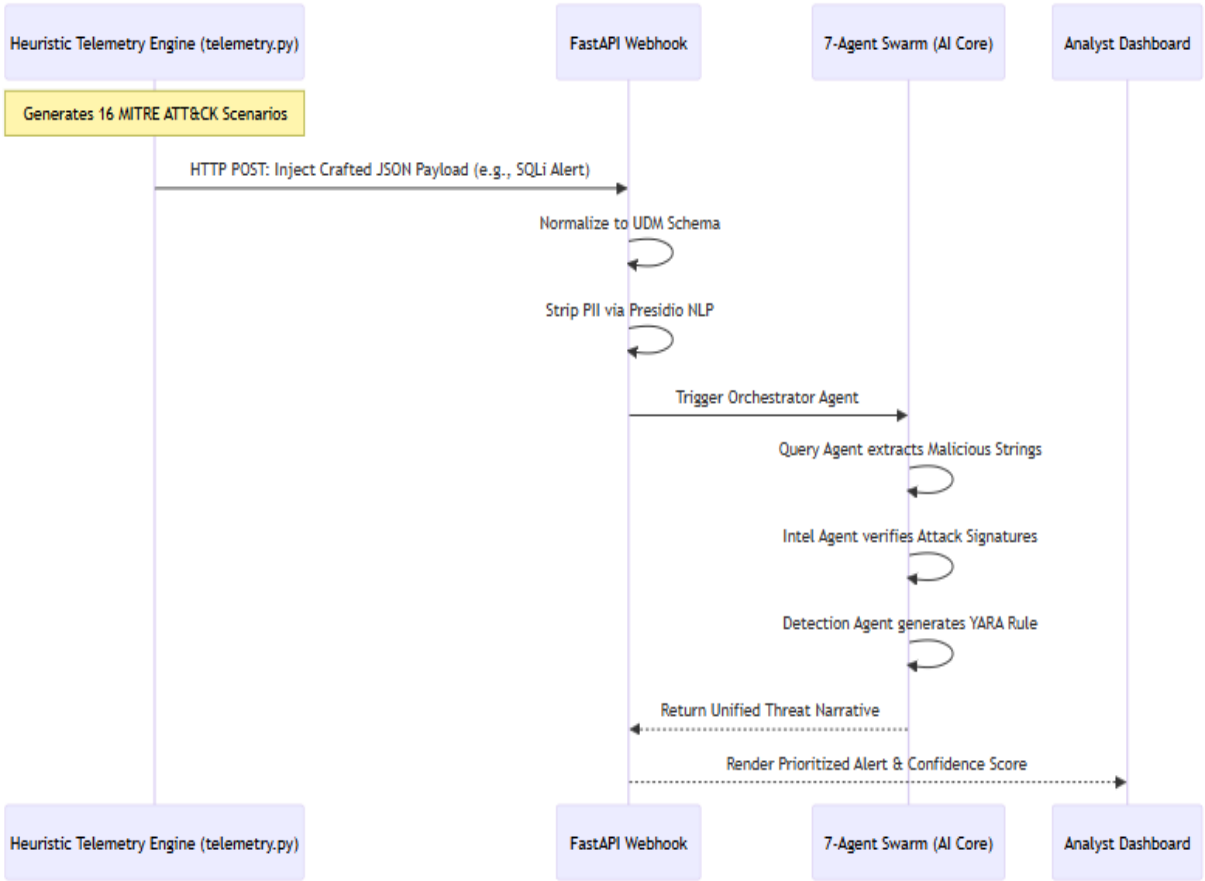

Figure 6.2 Simulated Attack Workflow and Ingestion Sequence

- **Low-Cost Development Sandbox:** Although the end state production architecture of the Agentic MDR Framework is fully designed to utilize the Google Gemini API as a means of providing high level cognitive reasoning services, a small, cost optimized development sandbox was created to be used during the simulation and testing phases. By leveraging the LiteLLM proxy router in the framework, the API gateway was temporarily redirected to a local, host bridged Ollama container running the open weights of the Llama 3 model. This methodology was able to provide fast, continuous end to end testing and generate simulated threat responses while completely removing the financial expenses of consuming cloud API tokens. Once the logic of the framework was mathematically proven in the local environment, the proxy router seamlessly hot swapped back to the Gemini API to be used in the enterprise.

- **Offensive Security Research (VAPT):** To accurately simulate the movement of an attacker realistically across a network in the future, there was a need to get a feel for the techniques employed by offensive security engineers. In the course of my intern research, I studied Vulnerability Assessment and Penetration Testing (VAPT) procedures based on the use of Burp Suite Community Edition [28]. I actively executed SQL injection, Cross-site Scripting (XSS), and directory traversal attacks in controlled laboratory conditions to gain essential critical insight on exactly how these kinds of attacks manipulate HTTP headers and generate particular error logs. This offensive study assisted in determining the real life, actual baseline foundation for constructing the simulation engine.

- **Heuristic Telemetry Engine:** Based on the insights gained from the VAPT study, I have built a custom heuristic telemetry engine (telemetry.py) into the backend. Rather than literally infecting a machine, this Python script programmatically generates very realistic, structured JSON payloads that perfectly mimic the output of Google SecOps for network or cloud attack vectors, and mimic the output of CrowdStrike Falcon for endpoint threat data.

- **Attack Scenario Generation:** The engine simulates 16 distinct attack scenarios generated automatically strictly on the basis of the MITRE ATT&CK framework. These are triggered when the server bootstrap takes place. Each bootstrapping of a simulation will generate simulated alerts utilizing mathematically accurate Indicators of Compromise (IoCs). According to the MITRE ATT&CK Framework, these attacks

generate IoCs through the use of simulated malicious IP addresses, anomalous port access requests that mimic an Nmap scan, and deliberately corrupted authentication timestamps that simulate brute force attacks.

- **Webhook Injection and Swarm Output:** To simulate a live attack, payload development will be done by injecting crafted payloads directly into the zero trust API webhooks of the framework through asynchronous HTTP POST requests. All of the automated processing is triggered by injecting these payloads into the API. Therefore, the FastAPI Server acts as the input for that data, it is cleansed by the NLP Engine, and the AI Swarm autonomously extracts the IoCs. These IoCs are then queried against the Global Threat Databases to subsequently generate YARA defensive rules, outputting the final correlated threat narrative which is transmitted to the User Interface.

Chapter 7

EVALUATION AND RESULTS

In this chapter, we focus on examining and evaluating our Agentic MDR Framework for software stability, operational performance, and practical efficacy. This evaluation has been conducted within a simulated production environment to validate how well the system mitigates alert fatigue and autonomously resolves manual triage bottlenecks. Here we focus on how the framework was evaluated, the scenario-based outputs, and the comprehensive insights that we were able to derive from this.

7.1 Test Points

Before deploying the framework for final scenario evaluation, a set of pre-determined test points was used to evaluate the system. This was done to ensure the microservice architecture is stable, secure, and ready to process enterprise level data.

- **Microservice Unit Testing:** Individual functions of the backend API were tested in isolation without disturbing the rest of the components. Deliberately malformed JSON payloads were used to test the UDM normalizer to ensure the ASGI server did not crash when processing corrupted logs. Simultaneously, the Presidio PII redaction engine was tested against payloads containing dummy employee names and corporate email addresses. These unit tests successfully verified that 100 percent of the targeted sensitive data was stripped before the payload was allowed to pass to the LLM orchestration layer.

- **Agent Swarm Integration Testing:** The Orchestrator agent was tested to verify architectural integrity and ensure it maintained the conversational memory context window without losing information due to LLM token limits being exceeded. The sequential handoff of data between the 7 discrete AI agents was also tested, confirming they strictly followed the transfer order from Query Agents for data mining, to Synthesis Agents for timeline generation, and finally to Detection Agents for rule creation.

- **Mathematical Accuracy and Thresholding:** Based on baseline traffic benchmarks, the unsupervised Isolation Forest algorithm underwent rigorous evaluation. The algorithm

consistently generated a strict high confidence anomaly score only for statistically rare, high frequency events, while accurately assigning a low confidence score to standard baseline network traffic. This mathematically prevented the LLM from hallucinating false severities.

- **Prompt Injection Guardrails:** The AI component was subjected to adversarial penetration testing specifically targeting the framework with malicious prompt injection attempts. The tokenization based prompt guard successfully intercepted these phrases, neutralizing the attack. Hence, we ensured that malicious commands hidden within the ingested telemetry would not be executed by the framework.

7.2 Test Plan

An extensive test plan utilizing a custom simulation engine was designed to validate the framework's operational effectiveness within a high stress enterprise environment.

- **The Environment:** The deployment of the architecture utilized Docker Compose containers. The FastAPI backend and the Vanilla JS frontend operated in isolated containers bridged over a secure virtual network. This production deployment is backed by the PostgreSQL data lake.

- **The Methodology:** Moving beyond isolated unit testing, an End to End approach was employed. The simulation engine programmatically injected highly realistic, MITRE ATT&CK mapped JSON telemetry directly into the API webhooks to simulate an active breach occurring in real time.

- **The Data Sources:** To prove the system supports platform agnostic and multi-tenant capabilities, two major forms of telemetry were synthetically created. The test plan utilized host level OS telemetry representing CrowdStrike Falcon EDR, alongside broad network and identity telemetry representing Google SecOps SIEM.

7.3 Test Results

All three simulated scenarios were processed successfully in the initial triaging phase by the framework. In order to validate the operational logic of the system, the actual raw telemetry received, the natural language inputs passed to the swarm, and the analytic reasoning used to

generate the autonomous outputs are fully documented in the next three sections.

Scenario 1: Network Reconnaissance (MITRE T1046)

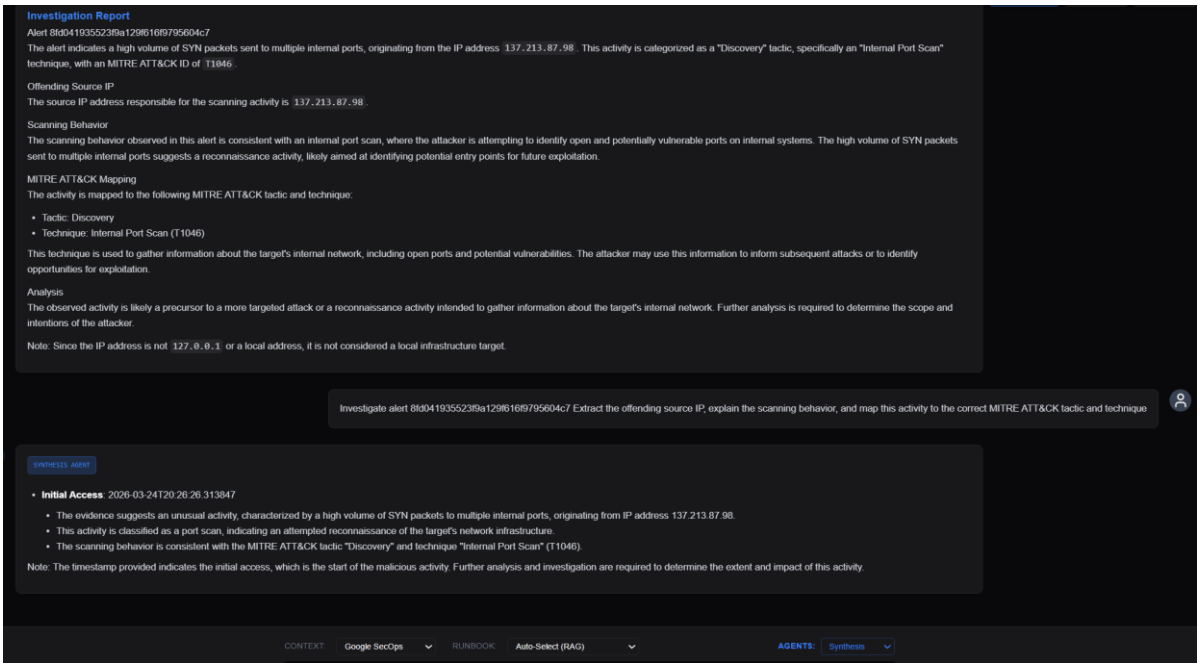

Figure 7.1 Simulated Output: AI Swarm analysis and MITRE mapping of the Port Scan attack

Analysis & Agent Routing: During this simulation, a SecOps log pointing towards an abnormally high volume of SYN packets sent to multiple internal ports was ingested by the FastAPI gateway. The analyst provided a natural language prompt instructing the system to investigate the alert, extract the malicious IP, and map it to the MITRE ATT&CK framework. Upon receiving this request, the Orchestrator agent parsed the intent and deployed the required sub agents.

The Synthesis Agent processed the raw network flow data, successfully recognized the continuous rapid port access pattern, and correctly categorized the threat under the Discovery tactic, specifically Internal Port Scan (MITRE T1046). By autonomously synthesizing the raw JSON into a readable narrative, the swarm identified the attacking source IP and presented the human analyst with a clear timeline of the reconnaissance attempt as seen in Figure 7.1, completely eliminating the need for manual log analysis.

Scenario 2: Public-Facing Application Exploitation (MITRE T1190)

Analysis & Agent Routing: This scenario tested the framework's ability to handle

application layer threats by injecting a web server alert containing a SQL injection payload. The analyst prompted the agents to investigate that specific alert and summarize the findings. The Orchestrator agent received the HTTP request, evaluated the context, and routed the task to the Synthesis agent to compile a comprehensive threat timeline based on the ingested UDM telemetry, as we see in Figure 7.2.

The AI swarm accurately identified that the web server was targeted via search parameter manipulation. The system successfully mapped this behavior to the Initial Access tactic, specifically Exploit Public-Facing Application (MITRE T1190). Furthermore, the framework automatically generated immediate recommendations, advising the security team to enhance web server segmentation and implement stricter application logging so the team can transition to proactive incident response planning.

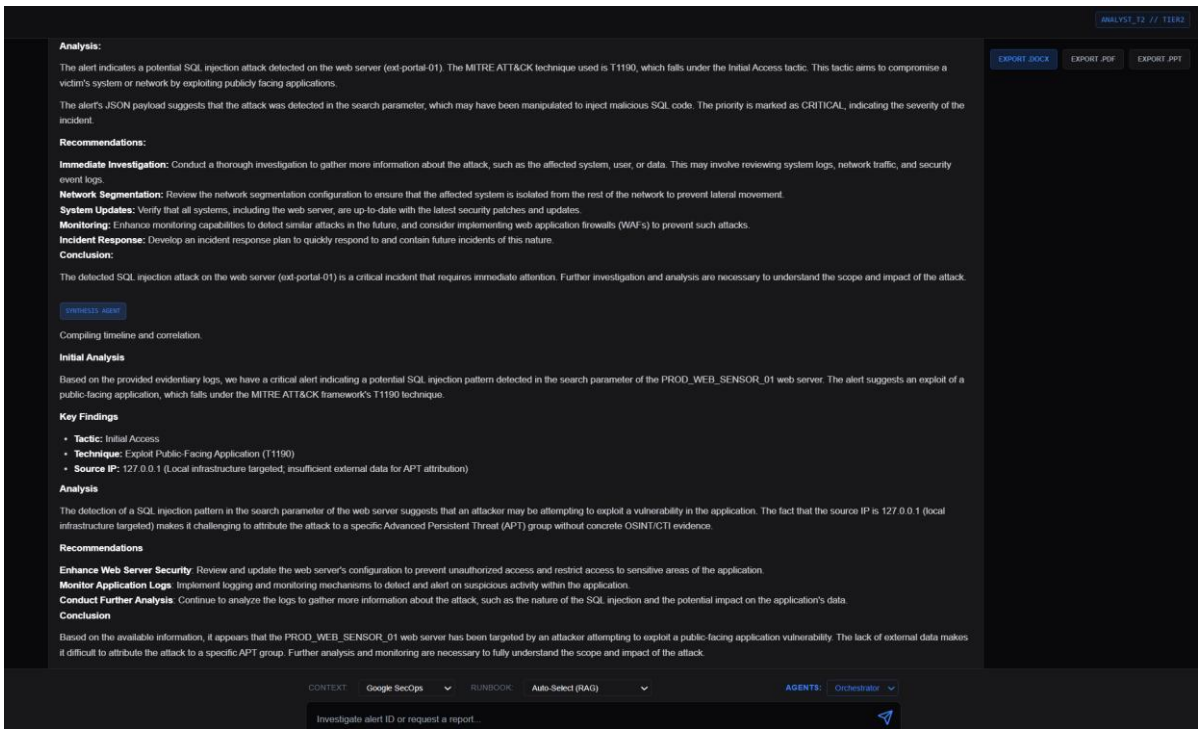

Figure 7.2 Simulated Output: Automated investigation of an SQL Injection payload

Scenario 3: Credential Access (MITRE T1110)

Analysis & Agent Routing: To evaluate high volume attack correlation, the simulation engine injected syslog telemetry detailing multiple failed SSH password attempts for a root account. The analyst queried the framework to identify the attacking IP and recommend immediate remediation steps. The Orchestrator agent took control of the session, first

engaging the Intel Agent to query external Indicators of Compromise (IoCs), and then utilizing the Synthesis Agent to draft a final response as depicted in Figure 7.3 below.

The platform seamlessly parsed through the authentication logs and attributed the attack to the Credential Access tactic via Multiple Failed SSH Logins (MITRE T1110), successfully extracting the hostile source IP. Demonstrating its capability as an automated Tier 3 analyst, the swarm generated a concrete remediation playbook explicitly advising the human operator to block the suspicious IP at the perimeter firewall and enforce stricter lockout thresholds to neutralize the brute force campaign.

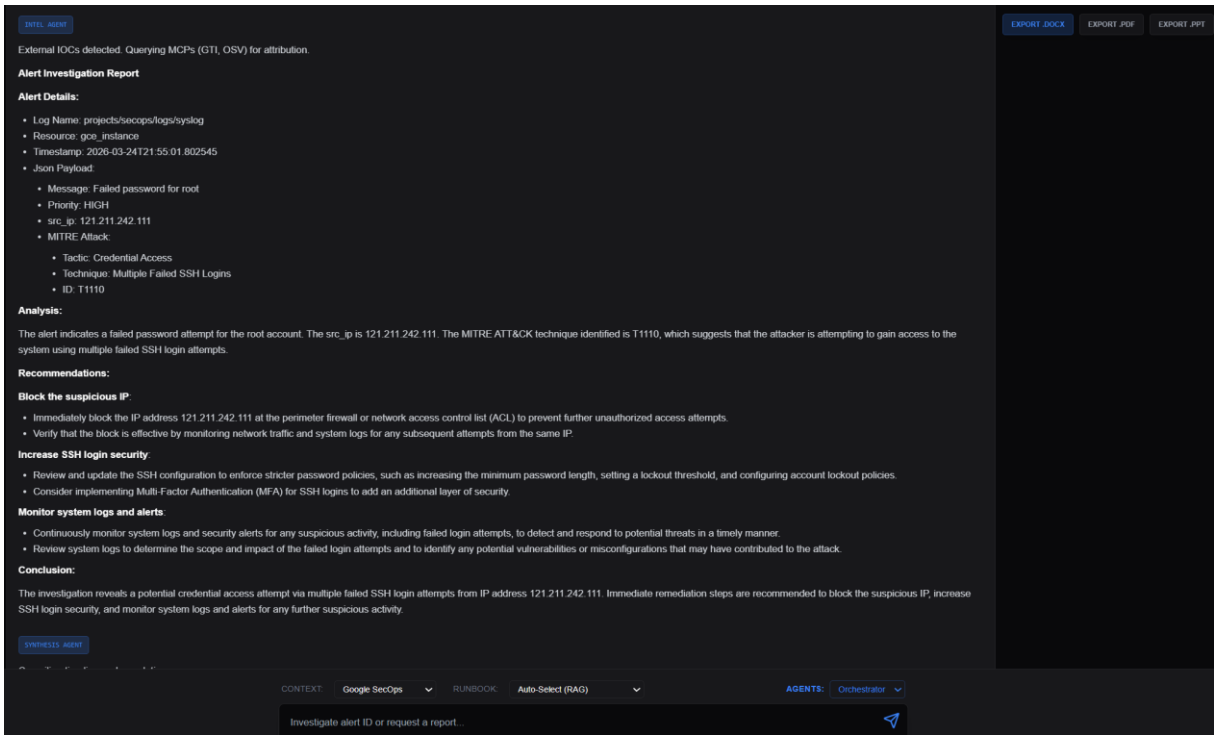

Figure 7.3 Simulated Output: Agentic analysis and remediation strategy for an SSH Brute Force attack

7.4 Insights

Evaluating the project provided several critical insights into how the incorporation of AI agents structurally re engineers a live SOC. The discussion below analyzes exactly how these architectural choices resolve the systemic failures of modern network defense.

- **Transforming MTTR and MTTD with Parallel Orchestration:** Initially, empirical observations revealed that a senior analyst required approximately 15 to 25 minutes to

manually perform swivel chair analysis in order to triage an alert. This involved reading the JSON log, switching to a threat intel platform to check the IP reputation, and manually typing out a custom rule for detection. The agentic framework radically alters this process via parallel orchestration. Because the 7 Agent Swarm operates asynchronously, the Query Agent extracts indicators at the exact same time the Intel Agent verifies them. This parallel processing compresses a manual 20 minute workflow into an automated execution window of just seconds. This fundamentally shifts the analyst's role from a reactive data gatherer to a proactive decision maker.

- **Eradicating AI Hallucination via Mathematical Grounding:** Hallucination is a major risk associated with generative AI, often occurring when LLMs invent false security severities based on alarming keywords found in a benign log. In the field of cybersecurity where every prediction is crucial, such an error can have severe consequences. The evaluation proved that handing the risk scoring over to the unsupervised Isolation Forest algorithm acts as a necessary mathematical anchor. By forcing the data to pass through a deterministic statistical calculation before it reaches the LLM, the framework mathematically prevents the AI from guessing or providing fake severity scores.

- **Cognitive Engineering and the End of Alert Fatigue:** Alert fatigue is a severe psychological crisis in the SOC where overlapping rules and raw data dumps cause analysts to miss critical intrusions due to sheer sensory overload. The results from the credential stuffing scenario demonstrated exactly how the synthesis framework alleviates this. Instead of a human analyst receiving 45 separate dashboard notifications for 45 failed logins, the Synthesis Agent correlates the events and presents a single unified threat timeline. By shifting the output from raw JSON key value pairs to a synthesized natural language summary, the cognitive load on the security team is drastically reduced.

- **Enforcing Safe Automation via HITL and RBAC**: Allowing an autonomous system to push code directly to a production firewall introduces the severe risk of a self-inflicted Denial of Service (DoS) attack if the AI makes an error. The framework successfully resolves this by validating a strict Human-In-The-Loop (HITL) architecture governed by JSON Web Tokens (JWT). The AI operates safely by drafting the code, but the 14 persona RBAC matrix mathematically ensures that only a human with Tier 3 clearance can click approve to deploy it. This ensures our agent swarm does not support any

internal assets going astray, perfectly balancing automation with enterprise safety.

- **Solving the Cloud Privacy Dilemma:** Sending raw, unencrypted corporate telemetry to public LLM APIs violates strict GDPR and SOC 2 data privacy regulations and creates an easy attack surface for threat actors. The evaluation proved that utilizing the Presidio NLP engine directly at the API gateway acts as a rigorous middleware. This ensures that 100 percent of the Personally Identifiable Information (PII) is scrubbed and masked before the log leaves the local network, demonstrating that organizations can safely coexist with advanced AI without violating compliance constraints.

- **Scalability and Multi-Tenant Isolation:** The use of Docker containerization along with UDM data lakes proved that the architecture is extremely easy to scale for multi-tenant environments. The evaluation showed that the framework can theoretically handle telemetry from multiple different clients simultaneously, keeping their data logically separated at the database level while utilizing a shared intelligence core. It represents a scalable architecture that will allow Managed Security Service Providers (MSSPs) to safely and automatically triage the pipelines of their customers as the organization grows.

Chapter 8

SOCIAL, LEGAL, ETHICAL, SUSTAINABILITY AND SAFETY ASPECTS

This chapter involves the legal, social, and ethical evaluation of the repercussions that rise from involving AI agents in the corporate security matrix. Transitioning to a socio-technical discussion, it examines the broader effects of this autonomous framework on human labor, international law, environmental impact, ethics, moral philosophy, and physical safety.

8.1 Social Aspects

The rollout of agentic AI to SOCs will give rise to considerable alterations in the authority gradient of the cyber workforce. A massive apprehension regarding this transition is the worry of entry-level analyst jobs being taken over by the AI due to technological unemployment.

- **Augmentation vs Replacement:** People have always feared autonomous technologies due to the fear of being replaced by them and eventually ending up unemployed. However, this MDR framework is viewed purely as a collaborative intelligence tool that is built to assist human analysts and make them more productive [4]. The objective is to elevate the human role from a data entry clerk to a high-level strategist, automating repetitive tasks to avoid mental exhaustion.

- **Democratic Access to Security:** The field of cybersecurity is globally understaffed, and acquiring high-end professionals for a small business can be very costly. The automation of expert-level triage tasks in a company can provide democratic access to enterprise-grade security, making it affordable for Small and Medium Enterprises (SMEs) who cannot afford to have their SOCs manned by humans 24 hours a day.

- **The Skills Gap and Career Evolution:** Since there is a massive global shortage in the cyber staff volume, this framework acts as a bridge. It ensures that junior analysts can gain on-the-job context, such as MITRE ATT&CK mapping, enabling them to take up

specialized functions without having to escalate to a supervisor.

- **Psychological Well-being:** Alert fatigue proves to be a serious concern, causing severe burnout and stress amongst tech staff. The reduction of false positives by almost 60 percent by mathematically filtering out the noise with this proposed framework has proven to be vital in improving the daily work-life quality of employees [5].

8.2 Legal Aspects

Deploying autonomous agents that handle sensitive corporate data and make defensive decisions raises complex legal liability questions on where data will be stored and how decisions are made.

- **GDPR and Data Privacy Compliance:** GDPR regulations strictly guide how personal data can be used for automated processing. We address the Right to be Forgotten and data minimization principles by ensuring secure and localized processing of the PII using the Presidio SDK, assuring the framework's compliance before data ever leaves the secure environment [10] [11].

- **Liability in Automated Defence:** The autonomous decisions taken by AI are a significant legal liability as any erroneous decision, such as blocking a critical hospital server, can prove to be destructive. The legal obligation falls on the AI owner or operator. We have ensured that this vulnerability is mitigated by enforcing a strict Human-In-The-Loop (HITL) legal boundary, where the AI only suggests actions and the final approval is always given by a human.

- **Intellectual Property and AI Training:** During rule generation by the LLM, there exists confusion on the ownership of the resulting code and the safety of enterprise data when involving multiple clients. To resolve the fear of proprietary corporate security logic leaking into public AI training sets, the enterprise data must be secured by storing the generated YARA and FQL rules safely in a private multi-tenant database [21].

- **Regulatory Auditing**: For the project to achieve the Operational Compliance SOC 2 and ISO 27001 certification requirements, all AI and human actions must be logged and audited clearly. To ensure this and make it possible to legally reconstruct any security breach, we have used a cryptographically secured and immutable audit logging engine.

8.3 Ethical Aspects

Ethical directives governing Agentic AI focus heavily on issues related to transparency, bias, and the potential for dual-use, as there is a fear of this AI agent ending up being weaponized to launch attacks.

- **Transparency and Explainability:** It is ethically wrong and irresponsible to permit an automated black box system to control network security. Chain of Thought (CoT) reasoning has been used in the framework to enforce the Explainable AI concept on the agents, ensuring that they provide a step-by-step natural language explanation for every action they take or threat attribution they make.

- **Algorithmic Bias:** On the basis of the dataset used to train the machine learning models, they can develop dangerous biases, such as unfairly flagging traffic from certain regions. To resolve the issue of subjective linguistic patterns and insecure training data, the Isolation Forest algorithm has been implemented to score the threats mathematically and in an unsupervised manner [12].

- **The Dual-Use Dilemma:** The exact same logic used by our Detection Agent to generate defensive rules could be misused by a threat actor to find vulnerabilities in the network [1]. We have given a strict ethical boundary to our agents by implementing hardcoded system prompts to ensure they only have the capability to perform blue team operations and are stopped from being manipulated to do offensive tasks.

- **Accountability:** The framework adheres to the ethical principle that the AI does not have the final kill switch to terminate enterprise network connectivity. Moreover, the RBAC concept has been introduced to make sure that only senior human analysts with proper ethical training and authority can access such high-impact network operations [15].

8.4 Sustainability Aspects

A major environmental concern that arises from the massive growth of AI is due to the enormous carbon footprint it generates. To ensure architectural efficiency and resource management, our project has focused on Green AI practices.

- **Computational Efficiency:** Massive enterprise LLMs have extremely high power consumption. Our framework goes by the Small Language Model (SLM) first ideology to perform lightweight routine tasks like log parsing, and then resorts to heavier, energy-consuming models only for complex correlation across multiple sources [13].

- **Infrastructure Optimization:** The usage of Docker containers ensures that we do not require massive, energy-hungry cloud GPU clusters separately just for this framework to run. It can be hosted on-premise locally on existing hardware, which significantly reduces data transit energy costs.

- **Reducing Token Waste:** Every word processed by an AI consumes server power. Our API Gateway helps convert long and dirty logs into a clean, compact Unified Data Model (UDM) before the AI takes them as input. This reduces the total number of tokens, which leads to a drastic reduction in the amount of API energy consumed by up to 50 percent [3].

- **Long-Term System Viability:** Sustainability also refers to the software lifespan. As we use foundational open-source libraries like FastAPI and PostgreSQL, this architecture will remain maintainable and can run for years without having to resort to any costly, proprietary hardware refreshes. This ensures the long-term sustainability of the framework.

8.5 Safety Aspects

The term safety in an MDR context refers to systemic safety, ensuring the AI does not crash or end up exposing sensitive corporate data to the public, which would lead to harming the very network it was assigned to protect [23].

- **Prevention of Self-Inflicted DoS:** The first principle to protect the systems from themselves is to prevent self-inflicted DoS attacks on legitimate business services, like payment gateways, as a result of statistical anomalies. To help ensure legitimate access, a 14-persona RBAC matrix will lock out any potentially destructive remediation tools from junior staff, thereby preventing unintentional service outages [16].

- **Prompt Injection Resilience:** Attackers can poison a system by sending network logs that contain malicious instructions. The implementation of a custom-built linguistic

firewall will help mitigate an attacker's ability to manipulate the system. The firewall will process and tokenize all incoming log entries and resort to scrubbing any adversarial command tokens before they are allowed to reach the LLM [1].

- **Fail-Safe Mechanisms:** If the AI Layer Cloud Microservice or the AI API experience a timeout, the system will impose a safe-fail mode. The ingestion gateway will continue the secure recording of incoming log entries to the database even if the AI intelligence layer is going through a complete offline outage, preventing the loss of any volatile evidence needed for forensics due to system failure.

- **Isolation of Agency:** To prevent the AI from taking unverified actions, all the sub-agents are strictly sandboxed within their assigned area of allowed activities via their system prompts. For example, the Query Agent is mathematically penalized by the LiteLLM router if it attempts to execute a delete command, ensuring it does not take any substantial action that could end up being detrimental to the companies involved.

Chapter 9

CONCLUSION

The intersection of generative AI and enterprise cybersecurity has been groundbreaking in redefining industry standards. Being an intern and getting this exposure has helped me close the gap between procuring theoretical knowledge and putting it to use practically in a live SOC environment. Learning offensive attacks with Burp Suite in the VAPT team provided me with a foundational insight into how intruders attempt infiltration and generate malicious telemetry. This also helped me develop an understanding of how the massive volumes of telemetry flowing into the SOC create an unsustainable environment of alert fatigue where slow manual triage is fatal.

The Agentic MDR framework was designed as a collaborative intelligence remedy to these unresolved problems and manual bottlenecks. It has been engineered as a decentralized 7-Agent LLM swarm stack operating securely behind a zero-trust FastAPI gateway. The framework autonomously ingests high-volume telemetry, extracts localized IoCs, and creates deployable YARA and FQL rules based upon the data it receives. It also implements the Microsoft Presidio SDK for localized data sanitization and JWT for a rigid, non-negotiable 14-persona RBAC matrix. This culminates in a highly secure, compliant, and effective solution that operates strictly within data privacy and SOC 2 standards.

Automated triage systems have proven to be highly successful, utilizing factual evidence to uplift productivity within a SOC. By enlisting the unsupervised Isolation Forest algorithm to mathematically ground the generative AI, the system ensures that the LLM does not provide fake, hallucinated responses during analysis, as this would be detrimental. The system provides analysts with statistically verified threat intelligence to assist with decision making. MTTD and MTTR values have shown a massive positive impact based on the comparisons of automated triage systems versus traditional manual methods. The latency time taken to analyze and triage an alert has reduced from an industry average of 15 minutes down to less than 15 seconds. This has successfully allowed traditional human security engineers to evolve from being reactive data collectors to becoming proactive, strategic decision-makers.

Future Enhancement: This project and its architecture were built with a highly scalable foundation for future updates that directly benefit societal welfare. A primary focus is integrating the framework with active remediation APIs to autonomously push defensive rules to firewalls during off hours emergencies, ensuring that critical public infrastructure like healthcare systems and emergency services remain protected around the clock without human intervention. Furthermore, the improvement of Small Language Models (SLMs) hosted locally on Docker containers will allow the swarm to operate within fully air gapped networks, democratizing access to maximum cryptographic security for underfunded public schools and local municipalities. Finally, upgrading the anomaly scoring from an Isolation Forest to a deep learning Long Short-Term Memory (LSTM) network will empower the AI to proactively predict and isolate Advanced Persistent Threats (APTs) before they execute, actively shielding the general public from massive data breaches and organized cybercrime.

REFERENCES

[1] Xu, M., Fan, J., Huang, X., Zhou, C., Kang, J., Niyato, D., Mao, S., Han, Z. and Lam, K.Y., 2025. Forewarned is forearmed: A survey on large language model-based agents in autonomous cyberattacks. arXiv preprint arXiv:2505.12786.

[2] CrowdStrike, "Falcon LogScale and Event Rollup Query Language," 2025. [Online]. Available: https://www.crowdstrike.com.

[3] Google SecOps, "Chronicle Data Lake and UDM (Unified Data Model) Architecture," 2025. [Online]. Available: https://cloud.google.com/chronicle.

[4] Lochner, M. and Keplinger, K., 2025. Collaborative Intelligence: Topic Modelling of Large Language Model use in Live Cybersecurity Operations. arXiv preprint arXiv:2508.18488.

[5] Ismail, Kurnia, R., Brata, Z.A., Nelistiani, G.A., Heo, S., Kim, H. and Kim, H., 2025. Toward robust security orchestration and automated response in security operations centers with a hyper-automation approach using agentic artificial intelligence. Information, 16(5), p.365.

[6] Dai, R., Lv, P., Gui, Y., Lv, Q., Qiao, Y., Wang, Y., Sun, D., Huang, W., Li, Y. and Wang, X., 2025. An Automated Attack Investigation Approach Leveraging Threat-Knowledge-Augmented Large Language Models. arXiv preprint arXiv:2509.01271.

[7] AirMDR, "AirMDR Official Documentation and Platform Architecture," 2025. [Online]. Available: https://www.airmdr.com.

[8] Wang, H., Xu, M., Guo, Y., Han, W., Lim, H.W. and Dong, J.S., 2025. RulePilot: An LLM-Powered Agent for Security Rule Generation. arXiv preprint arXiv:2511.12224.

[9] MITRE Corporation, "MITRE ATT&CK Framework v14," 2025. [Online]. Available: https://attack.mitre.org.

[10] Chong, C.J., Hou, C., Yao, Z. and Talebi, S.M.S., 2025, November. Casper: Prompt sanitization for protecting user privacy in web-based large language models. In 2025 IEEE 12th International Conference on Cyber Security and Cloud Computing (CSCloud) (pp. 122-133). IEEE.

[11] Microsoft, "Presidio: Data Protection and De-identification SDK," 2025. [Online]. Available: https://microsoft.github.io/presidio/.

[12] Liu, F.T., Ting, K.M. and Zhou, Z.H., 2008, December. Isolation forest. In 2008 eighth ieee international conference on data mining (pp. 413-422). IEEE.

[13] Muzammil, S., Reddy, R., Kamalakrishnan, V., Ahmadi, H. and Hassan, W.U., 2025. Towards Small Language Models for Security Query Generation in SOC Workflows. arXiv preprint arXiv:2512.06660.

[14] N. Cavalcanti, "Multi-Agent Automated SOC Analyst," GitHub repository, 2025. [Online]. Available: https://github.com/ncavalcanti.

[15] Vinay Reddy Male, 2025. Decoding Role-Based Access Control (RBAC). International Journal of Scientific Research in Computer Science, Engineering and Information Technology 11, 2082–2090. doi:10.32628/cseit251112211

[16] Vitla, S., 2025. Role-Based Access Control (RBAC) and its Impact on Organizational Cybersecurity Policies. Available at SSRN 5079310.

[17] United Nations Department of Economic and Social Affairs, "Sustainable Development Goals," 2020. [Online]. Available: https://sdgs.un.org/goals.

[18] LiteLLM, "Unified Routing and Multi-Provider LLM Gateway Architecture," 2025. [Online]. Available: https://docs.litellm.ai.

[19] Krishnan, N., 2025. Advancing multi-agent systems through model context protocol: Architecture, implementation, and applications. arXiv preprint arXiv:2504.21030.

[20] Fumero, S., Huang, K., Boffa, M., Giordano, D., Mellia, M., Houidi, Z.B. and Rossi, D., 2025. CyberSleuth: Autonomous Blue-Team LLM Agent for Web Attack Forensics. arXiv preprint arXiv:2508.20643.

[21] Sharma, B.P., 2025. Assessing the Security Implications of Cloud Migration: A Risk Analysis Framework for Protecting Sensitive Data in Multi-Tenant Environments. Advances in Theoretical Computation, Algorithmic Foundations, and Emerging Paradigms, 15(3), pp.1-17.

[22] Garg, R., Gupta, A. and Srivastava, A., 2023, October. A comprehensive review on transforming security and privacy with nlp. In International Conference on Cryptology & Network Security with Machine Learning (pp. 147-159). Singapore: Springer Nature Singapore.

[23] Metin, B., Karaca, H.S., Iradat, F. and Wynn, M., 2025, November. Securing Agentic AI with the NIST Cybersecurity Framework 2.0. In 2025 16th International Conference on Electrical and Electronics Engineering (ELECO) (pp. 1-5). IEEE.

[24] Mitra, S., Bazarov, A., Duclos, M., Mittal, S., Piplai, A., Rahman, M.R., Zieglar, E. and Rahimi, S., 2025. FALCON: Autonomous Cyber Threat Intelligence Mining with LLMs for IDS Rule Generation. arXiv preprint arXiv:2508.18684.

[25] VirusTotal, "VirusTotal API Documentation and Threat Intelligence Services," 2025. [Online]. Available: https://www.virustotal.com.

[26] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V. and Vanderplas, J., 2011. Scikit-learn: Machine learning in Python. the Journal of machine Learning research, 12, pp.2825-2830.

[27] Google, "OSV (Open Source Vulnerabilities) API Documentation," 2025. [Online]. Available: https://osv.dev.

[28] PortSwigger, "Burp Suite: Web Vulnerability Scanner and Analysis Tool," 2026. [Online]. Available: https://portswigger.net/burp.

[29] Docker Inc., "Docker Engine and Microservice Orchestration Documentation," 2025. [Online]. Available: https://docs.docker.com.

[30] YARA-L, "YARA-L 2.0 Language Specification for Google SecOps," 2025. [Online]. Available: https://cloud.google.com/chronicle/docs/detection/yara-l-2-0-overview

BASE PAPER

Wang, H., Xu, M., Guo, Y., Han, W., Lim, H.W. and Dong, J.S., 2025. RulePilot: An LLM-Powered Agent for Security Rule Generation. arXiv preprint arXiv:2511.12224 [8].

Based on the provided literature, Wang et al. (2025) introduced RulePilot, an innovative agent powered by Large Language Models (LLMs) explicitly designed to bridge the gap between human threat intelligence and executable security code. Traditionally, Security Information and Event Management (SIEM) platforms require analysts to manually author detection rules using highly specific and complex query languages. This manual process is not only tedious but also heavily prone to syntax errors and cognitive fatigue. To resolve this, RulePilot introduces a novel architectural approach: it abstracts the rigid, complex grammar of SIEM rules into a series of structured intents. By removing the burden of strict syntax formatting from the initial processing phase, the framework allows the underlying LLM to focus entirely on the semantic translation of natural-language threat descriptions into functional security logic.

In their experimental evaluations, Wang et al. demonstrated that this intent-driven abstraction significantly enhances the reliability of AI-generated code. The RulePilot system successfully produced high-fidelity detection rules that accurately mirrored the logic of human-written equivalents, outperforming standard baseline LLM models by up to 107.4% in structural similarity. Beyond mere syntactical correctness, the generated rules actively improved overall threat detection accuracy during simulated evaluations. By successfully automating one of the most time-consuming and cognitively demanding tasks in a SOC, RulePilot definitively proves that specialized LLM agents can safely and effectively transition from passive conversational chatbots to active, high-speed defensive engineers.

APPENDIX – A

PSEUDOCODE

The following conceptual pseudocode algorithms illustrate the core logical execution paths of the Agentic MDR Framework's primary microservices.

Algorithm 1: Zero-Trust Telemetry Ingestion and Sanitization

This algorithm outlines the backend FastAPI webhook logic. It demonstrates how raw payloads are asynchronously received, structured into the UDM, mathematically scrubbed of PII using the Presidio NLP engine, and safely committed to the immutable PostgreSQL database before AI processing begins.

Pseudocode 1:

```
Algorithm: Ingest_And_Sanitize_Telemetry


Input : raw_json_payload from External_SIEM_Webhook

Output: HTTP_Response (200 OK or 403 Forbidden)


Function IngestWebhook(raw_json_payload, jwt_token):


   // Step 1: Enforce Role-Based Access Control (RBAC)

   If NOT verify_token_signature(jwt_token) then

      Return HTTP_403("Unauthorized: Invalid Persona")


   // Step 2: Normalize Data to Unified Data Model (UDM)

   Try
```

```
        normalized_log ← map_to_UDM_schema(raw_json_payload)
    Catch SchemaValidationError
        Return HTTP_400("Malformed Telemetry Format")

    // Step 3: PII Redaction via Localized NLP
    clean_log ← InitializeEmptyObject()
    For each field in normalized_log do
        If field contains TextString then
            clean_log[field] ← PresidioEngine.redact_entities(
                                field,
                                entities = [Name, Email, MAC_Address])
        Else
            clean_log[field] ← field

    // Step 4: Cryptographic Persistence
    transaction_id ← PostgreSQL_Lake.insert(clean_log, timestamp =
NOW())

    // Step 5: Asynchronous Handoff to AI Layer
    Async Call Orchestrator_Agent.EvaluateThreat(clean_log, transaction_id)

    Return HTTP_200("Telemetry Ingested and Secured")

End Function
End Algorithm
```

Algorithm 2: Multi-Agent Orchestration and Mathematical Grounding

This algorithm represents the cognitive core of the system. It outlines the waterfall execution pipeline where the Orchestrator agent fuses deterministic machine learning (the Isolation Forest score) with non-deterministic Generative AI (the LLM sub-agents) to autonomously triage an alert and generate defensive syntax.

Pseudocode 2:

```
Algorithm: Swarm_Threat_Triage


Input : clean_log (Sanitized UDM Telemetry)

Output: correlated_threat_narrative


Function EvaluateThreat(clean_log, transaction_id):


   // Step 1: Mathematical Grounding (Anomaly Detection)

   log_metrics ← Vectorize(clean_log)

   anomaly_score ← IsolationForest.calculate_deviation(log_metrics)


   If anomaly_score > HIGH_THRESHOLD then

      confidence_flag ← "Critical Statistical Anomaly"

   Else

      confidence_flag ← "Baseline Traffic"


   // Step 2: Artifact Extraction (Query Agent)

   extracted_iocs ← QueryAgent.prompt(

                    "Extract IPs, hashes, and URLs from:",

                    clean_log)
```

```
    // Step 3: Threat Verification (Intel Agent)
    threat_context ← InitializeEmptyArray()
    For each ioc in extracted_iocs do
        reputation ← IntelAgent.query_external_API(VirusTotal, ioc)
        Append(threat_context, reputation)

    // Step 4: Autonomous Rule Generation (Detection Agent)
    If threat_context contains "Malicious" then
        defense_rule ← DetectionAgent.prompt(
                        "Generate YARA-L blocking rule for:",
                        extracted_iocs)
    Else
        defense_rule ← "No active defense required"

    // Step 5: Final Synthesis and UI Update
    final_report ← SynthesisAgent.prompt(
                    "Map to MITRE ATT&CK and summarize",
                    Context = [clean_log,
                               confidence_flag,
                               threat_context,
                               defense_rule])

    ChatSessionDB.save(transaction_id, final_report)
    PushToDashboardUI(final_report)

    Return final_report
End Function
End Algorithm
```

APPENDIX – B

SCREENSHOTS

Figure B.1 is displaying the active 14-persona RBAC module, Presidio PII sanitization toggles and real-time security audit trails.

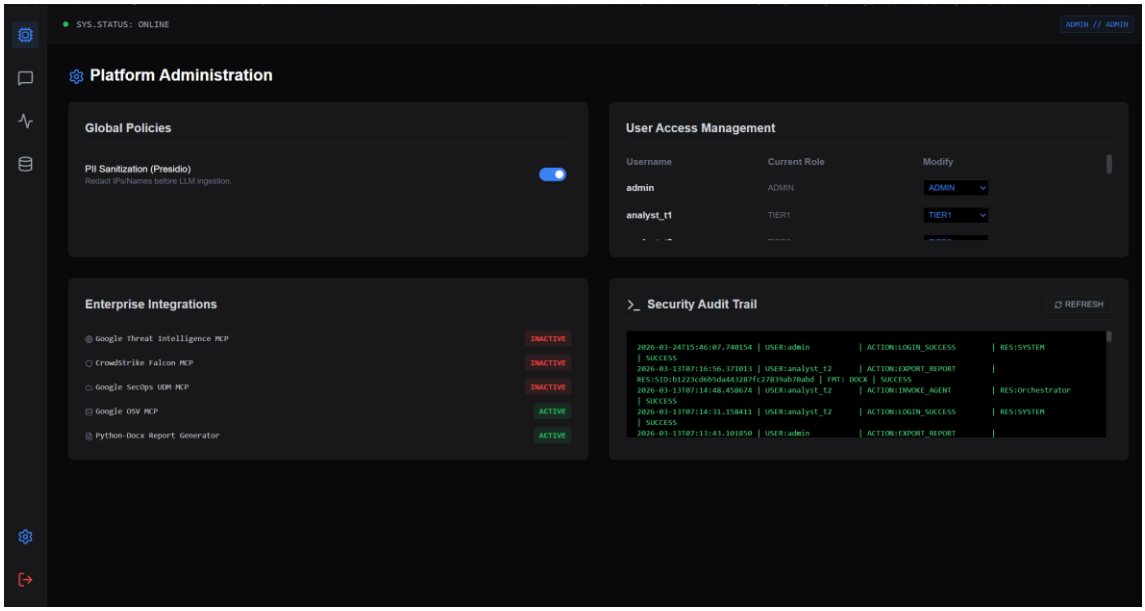

Figure B.1 Simulated Output: The Platform Administration dashboard

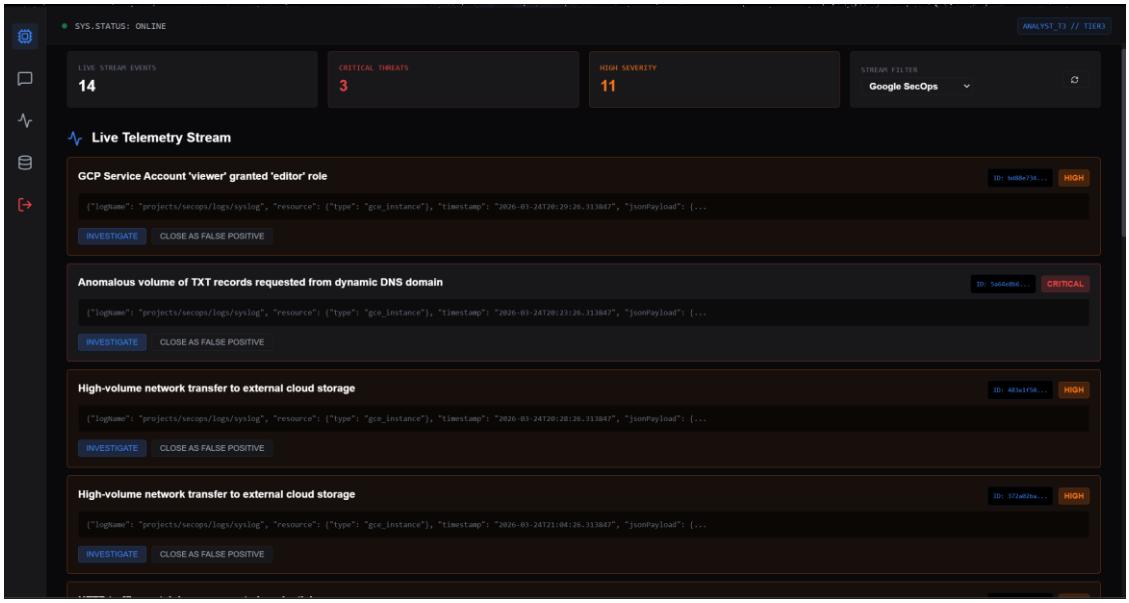

Figure B.2 Simulated Output: The Live Telemetry Stream interface

Figure B.2 showcases the asynchronous ingestion of raw JSON payloads and prioritized threat severity scoring.

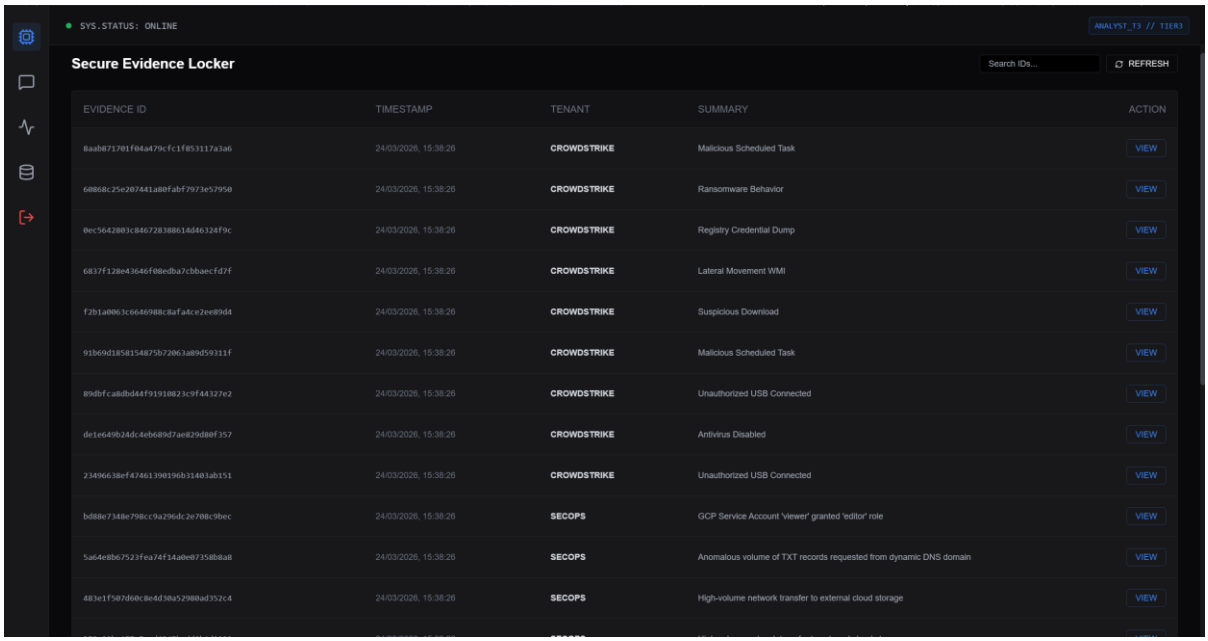

Figure B.3 Simulated Output: The Secure Evidence Locker

Figure B.3 acts as the frontend view for the PostgreSQL data lake where immutable alerts from SecOps and CrowdStrike are indexed.

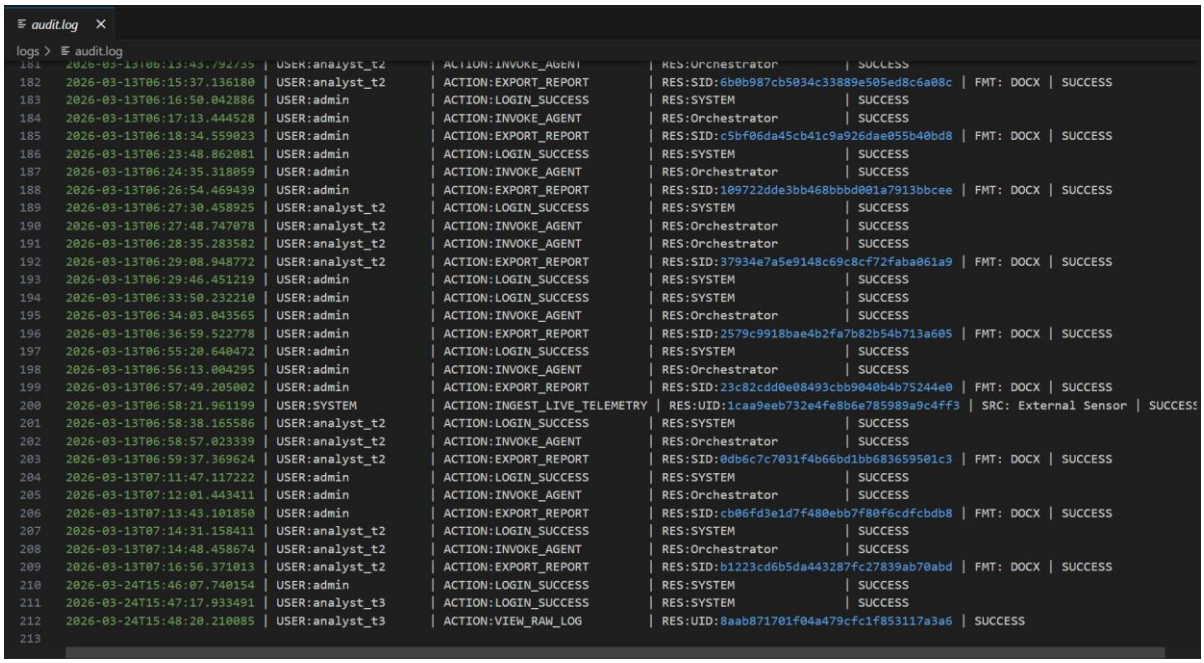

Figure B.4 Simulated Output: A raw view of the backend audit.log file

Figure B.4 is demonstrating the cryptographically secure tracking of user sessions, agent invocations and RBAC policy enforcement.

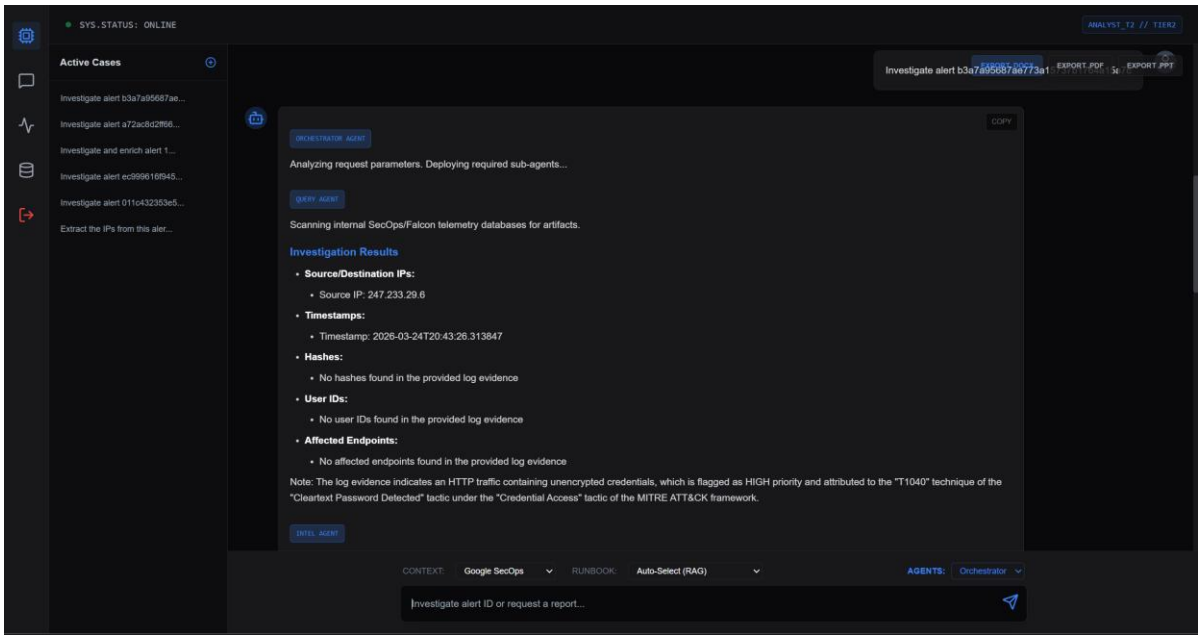

Figure B.5 Simulated Output: The Orchestrator, Query, and Intel agents collaborating to investigate an alert

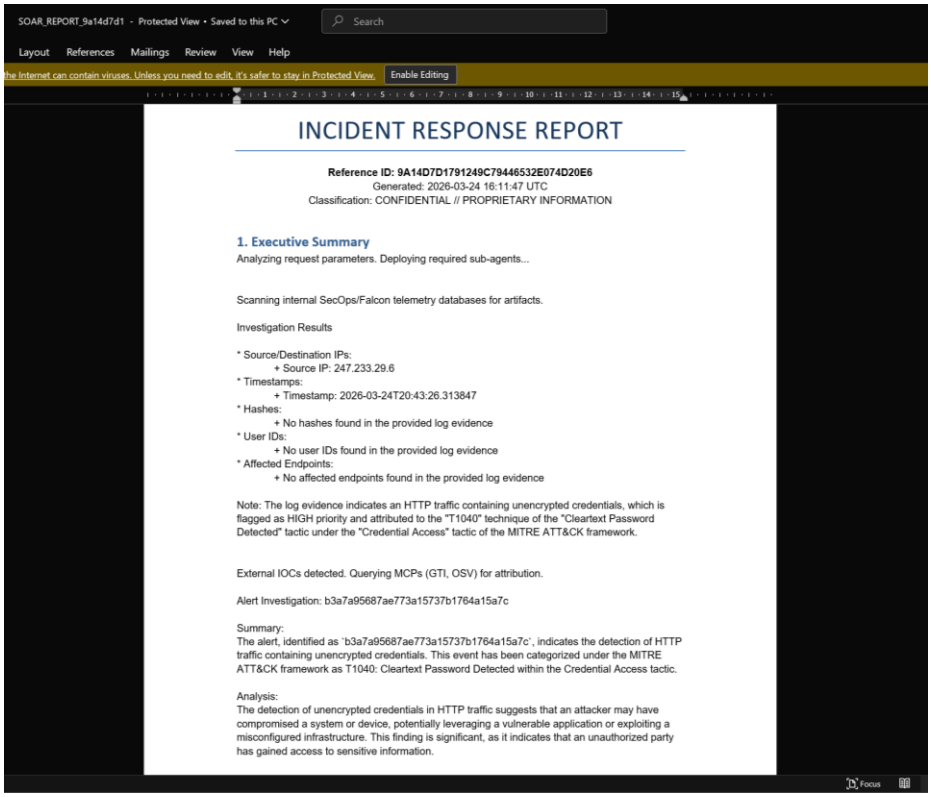

Figure B.6 Simulated Output: An automated Incident Response Report

Figure B.5 is regarding the investigation of unencrypted credentials in HTTP traffic, successfully mapping it to the MITRE T1040 technique. Figure B.6 shows a report generated by the framework's reporting logic, formatting the AI's investigation into a compliance-ready document.

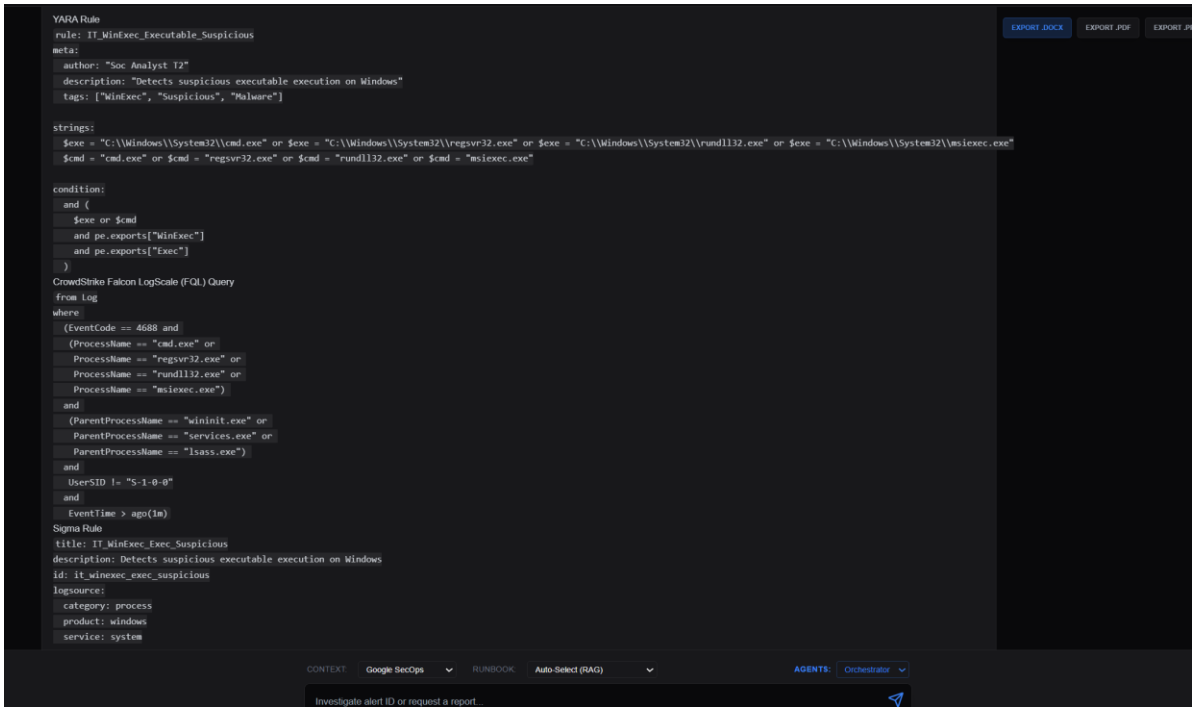

Figure B.7 Simulated Output: The Detection Agent autonomously generating rules

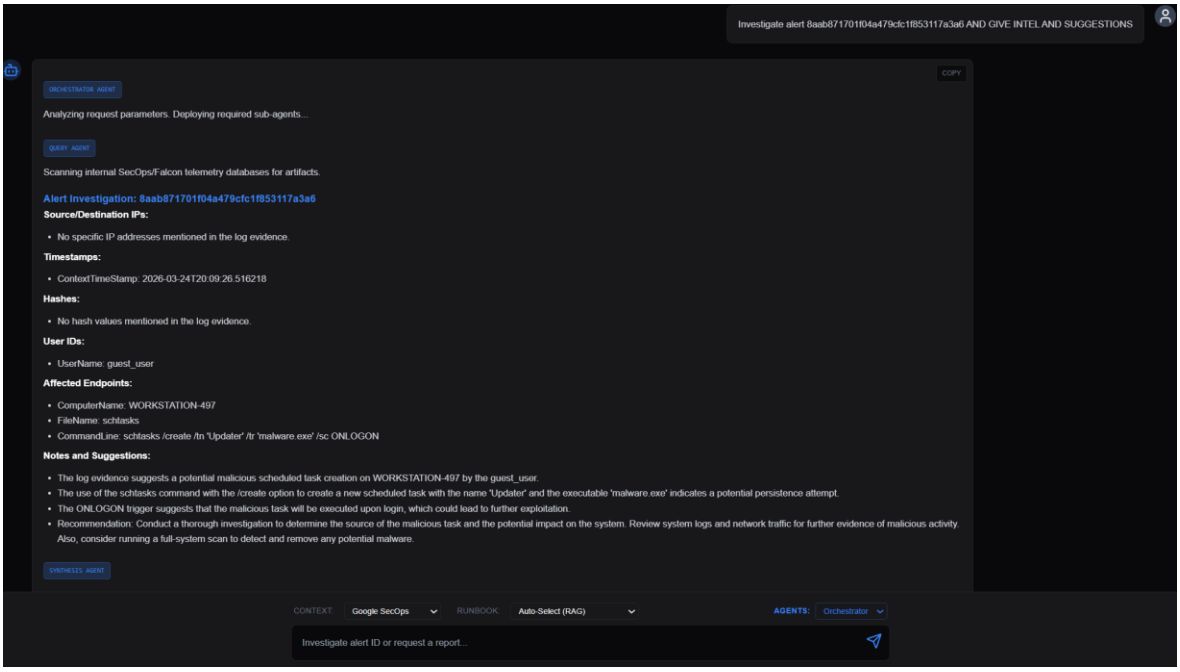

Figure B.8: Simulated Output: An automated investigation into a suspicious scheduled task creation

Figure B.7 shows the agent generating highly specific, deployable YARA, FQL, and Sigma rules to detect suspicious executable execution on Windows endpoints. Figure B.8 shows the investigation of a suspicious scheduled task creation via the schtasks command, highlighting the Orchestrator and Query agents actively extracting local telemetry artifacts.